Universidad Tecnológica de Pereira

Facultad de Ingenierías

Programa de Ingeniería de Sistemas y Computación

Procesador RISC-V segmentado RV32IM con pantalla de visualización de estados en FPGA DE1-SoC

ANTEPROYECTO

Presentado por:

Juan Esteban Cardona Blandón

Código: 1113858851

Pereira, Colombia

1 de marzo de 2026

Índice

1. Introducción **4**

2. Planteamiento del Problema **5**

2.1. Descripción del problema 5
2.2. Formulación del problema 5
2.3. Antecedentes 5
2.4. Causas 6
2.5. Definición del problema 11
2.6. Consecuencias 11

3. Justificación **12**

3.1. Justificación científica y tecnológica 12
3.2. Justificación educativa y social 12
3.3. Justificación Técnica 12

4. Objetivos **13**

4.1. Objetivo General 13
4.2. Objetivos Específicos 13

5. Metodología **14**

5.1. Marco metodológico 14
5.1.1. Descripción de tratamientos 14
5.1.2. Unidad experimental 14
5.1.3. Número de unidades y respaldo estadístico 14
5.1.4. Diseño experimental y asignación 15
5.1.5. Procedimiento y registro de información 15
5.1.6. Variables de respuesta e independientes 16
5.1.7. Criterios para suspensión o replanteamiento del proyecto 17
5.1.8. Análisis y evaluación de la hipótesis 18
5.1.9. Evaluación de la hipótesis de trabajo 18
5.2. Hipótesis 19
5.3. Actividades 19

6. Aportes **21**

7. Marco referencial **22**
7.1. Marco teorico . . . 22
7.1.1. Arquitectura de Computadores y Pipeline . . . 22
7.1.2. Diseño Digital y Arquitectura RISC-V . . . 22
7.1.3. Especificación oficial RISC-V . . . 23
7.1.4. Métricas de rendimiento en arquitectura de computadores . . . 24
7.1.5. Síntesis en FPGA y análisis de timing . . . 24
7.2. Estado del arte . . . 25
7.2.1. Introducción . . . 25
7.2.2. Implementaciones monociclo de RV32I/RV32IM en FPGA . . . 26
7.2.3. Procesador RV32I monociclo en Intel Cyclone V DE1-SoC . . . 26
7.2.4. Procesador RV32I compacto con verificación rigurosa . . . 27
7.2.5. Síntesis comparativa de implementaciones monociclo . . . 27
7.2.6. Implementaciones pipeline de RV32I/RV32IM en FPGA . . . 28
7.2.7. Pipeline RV32I optimizado con técnicas avanzadas de resolución de hazards . . . 29
7.2.8. Catálogo y evaluación in-hardware de cores RISC-V open-source . . . 29
7.2.9. Síntesis comparativa de implementaciones pipeline . . . 30
7.2.10. Técnicas de resolución de hazards en arquitecturas pipeline . . . 31
7.2.11. Estudios comparativos multi-arquitectura . . . 32
7.2.12. Metodologías de verificación funcional . . . 33
7.2.13. CoreMark como benchmark estándar para procesadores embebidos . . . 34
7.2.14. Identificación de la brecha en la literatura . . . 34
7.2.15. Conclusiones del estado del arte . . . 36

8. Marco conceptual **38**
8.1. Arquitectura de computadores . . . 38
8.2. Instruction Set Architecture (ISA) . . . 38
8.3. RISC-V y extensión M (RV32IM) . . . 39
8.3.1. RISC-V como arquitectura abierta . . . 39
8.3.2. Configuración RV32IM empleada en el proyecto . . . 39
8.4. Pipeline de procesadores . . . 40
8.4.1. Definición y etapas . . . 40
8.4.2. Ecuación de rendimiento y comparación monociclo vs. pipeline . . . 40
8.5. Hazards en pipelines de procesadores . . . 41
8.5.1. Tipos de hazards . . . 41

8.5.2. Impacto de los hazards en el CPI 43
8.5.3. Field-Programmable Gate Array (FPGA) 43
8.5.4. Componentes FPGA relevantes 43
8.5.5. FPGA DE1-SoC (Cyclone V 5CSEMA5F31C6) 44
8.6. SystemVerilog y diseño RTL 44
8.6.1. Nivel RTL 44
8.6.2. Proceso de síntesis sobre FPGA 45
8.7. Verificación funcional y suites de prueba 45
8.7.1. riscv-tests como suite de conformidad 45

9. Presupuesto **46**

1. Introducción

La literatura reciente recoge numerosas implementaciones de procesadores RISC-V RV32I y RV32IM en FPGAs, tanto en variantes de ciclo único como con pipeline de cinco etapas, con resultados de frecuencia máxima, CPI y rendimiento en benchmarks debidamente cuantificados. Sin embargo, la mayoría de estos trabajos evalúan únicamente una de las arquitecturas o utilizan plataformas y metodologías heterogéneas, de modo que no existe evidencia empírica directa que compare, bajo condiciones experimentales controladas, un procesador RV32IM monociclo y su variante segmentada implementados en la misma FPGA.

El presente proyecto aborda esta brecha mediante el diseño e implementación en SystemVerilog de un procesador RISC-V RV32IM en dos microarquitecturas alternativas (monociclo y pipeline de cinco etapas) sobre la tarjeta DE1-SoC, incorporando técnicas de resolución de hazards de datos y de control y utilizando suites de prueba estandarizadas y un modelo de referencia funcional para garantizar la corrección respecto a la especificación de la ISA. A partir de estas implementaciones se realiza una comparación experimental cuantitativa que incluye métricas como frecuencia máxima, CPI efectivo, throughput y rendimiento por MHz, así como la utilización de recursos lógicos y bloques DSP de la FPGA.

El objetivo general del trabajo es determinar, en un entorno educativo concreto, en qué medida la microarquitectura pipeline RV32IM implementada en FPGA mejora el rendimiento frente a un procesador monociclo equivalente y cuál es el coste asociado en términos de área y complejidad. De forma más específica, se plantean metas como la especificación modular de ambas microarquitecturas en SystemVerilog, la verificación funcional exhaustiva del conjunto de instrucciones RV32IM y la definición de un protocolo de medición homogéneo que permita comparar de manera rigurosa y reproducible los *trade-offs* reales entre las dos arquitecturas estudiadas.

2. Planteamiento del Problema

2.1. Descripción del problema

La segmentación es la metodología común para mejorar el throughput de los procesadores al superponer la ejecución de instrucciones y facilitar ciclos de reloj más reducidos (Patterson & Hennessy, 2017). Sin embargo, en aplicaciones reales, la ganancia teórica puede estar influenciada por hazards de control y de datos, así como por la latencia de operaciones complejas (extensión M) y el costo adicional de la lógica para la resolución de hazards, lo que incrementa el CPI efectivo y utiliza recursos de la FPGA (Chu, 2011; Waterman & Asanović, 2019). Hay una falta de investigaciones experimentales consistentes que contrasten, en la misma plataforma FPGA y utilizando el mismo protocolo de medición, un núcleo RV32IM realizado tanto en su versión de ciclo único como en pipeline de cinco etapas. Sin esa comparación controlada, no existe evidencia replicable para determinar si, en FPGAs de capacidad media (por ejemplo. DE1-SoC), la complejidad extra del pipeline compensa el coste en términos de eficiencia, superficie y energía. Para abordar esa brecha, esta investigación ejecutará ambas microarquitecturas en la DE1-SoC, evaluará Fmax tras place-and-route, CPI en hardware y verificará la corrección funcional a través de riscv-tests complementados con una pantalla de visualización que posibilitará la inspección ciclo a ciclo de registros interetapa y señales de hazard.

2.2. Formulación del problema

Pregunta principal: ¿La implementación de una microarquitectura RISC-V RV32IM segmentada en cinco etapas sobre FPGA mejora el rendimiento, en términos de throughput y frecuencia, frente a un procesador monociclo equivalente, manteniendo la corrección funcional?

2.3. Antecedentes

La técnica de segmentación o pipelining se consolidó desde la década de 1960 como mecanismo esencial para aumentar el throughput de procesadores mediante la superposición de la ejecución de instrucciones (etapas IF, ID, EX, MEM, WB) y sigue siendo la estrategia predominante en diseños RISC educativos y comerciales (Patterson & Hennessy, 2017). Sin embargo, investigaciones clásicas y contemporáneas han mostrado que el beneficio teórico del pipeline depende fuertemente de las dependencias entre instrucciones (hazards) y de las técnicas de mitigación aplicadas; sin una estrategia efectiva de forwarding, stalling y gestión

de saltos, el CPI efectivo puede aumentar notablemente, reduciendo el speed-up práctico del pipeline (Smith & Pleszkun, 1988; Wall, 1991).

RISC-V, una ISA abierta y modular desarrollada en Berkeley, ha impulsado una ola de implementaciones académicas y experimentales en FPGA por su accesibilidad y flexibilidad para extender la ISA (Waterman & Asanović, 2019). La configuración RV32IM es de particular interés en aplicaciones embebidas que requieren operaciones aritméticas aceleradas por hardware; sin embargo, la inclusión de instrucciones M introduce latencias y caminos críticos que afectan de manera distinta a arquitecturas monociclo y pipeline.

Las FPGAs constituyen una plataforma para prototipado y verificación in-hardware de microarquitecturas debido a su reconfigurabilidad y tiempos de iteración acortados (Chu, 2011). En la literatura reciente se han reportado implementaciones RISC-V tanto monociclo como pipelined en distintas familias de FPGA, con resultados muy variables en Fmax, CPI y CoreMark/MHz (Heinz et al., 2019; NRP, 2024). No obstante, la mayoría de los estudios presentan una de estas tres limitaciones:

1. Implementaciones aisladas (solo monociclo o solo pipeline)
2. Empleo de diferentes plataformas FPGA (Xilinx vs. Intel/Altera)
3. Evaluación con protocolos experimentales no homogéneos que impiden una comparación directa.

Por tanto, en la literatura revisada se identifica una brecha: no existen, en fuentes accesibles, estudios que implementen ambas microarquitecturas del mismo núcleo RV32IM, sintetizadas en la misma plataforma FPGA, evaluadas con el mismo flujo de síntesis, como lo es Quartus Prime, las mismas suites de verificación, como riscv-tests, y protocolos de medida comparables. Sin esta comparación controlada, falta evidencia empírica reproducible para decidir si, en FPGAs de capacidad moderada, el overhead del pipeline compensa el potencial aumento de frecuencia y throughput.

2.4. Causas

La ausencia de comparaciones experimentales controladas entre arquitecturas monociclo y pipeline de procesadores RISC-V RV32IM implementados en la misma plataforma FPGA no es accidental, sino resultado de múltiples factores técnicos, metodológicos y prácticos que dificultan la realización de este tipo de estudios. A continuación se analizan las causas principales que han perpetuado esta brecha en la literatura científica.

1. La implementación de un procesador funcional completo en FPGA requiere un esfuerzo de desarrollo significativo que incluye diseño RTL, verificación funcional, síntesis, optimización de timing, y depuración iterativa (Chu, 2011). Cada arquitectura (monociclo y pipeline) presenta desafíos técnicos específicos que multiplican el tiempo de desarrollo total.

   Para arquitecturas monociclo, aunque conceptualmente más simples, el diseño debe manejar un camino de datos extenso donde todas las etapas (fetch, decode, execute, memory, writeback) operan secuencialmente en un único ciclo largo. Esto resulta en caminos críticos combinacionales extensos que complican el análisis de timing y requieren optimización cuidadosa para alcanzar frecuencias razonables (Harris & Harris, 2021). La unidad de control debe generar todas las señales necesarias basándose únicamente en el opcode de la instrucción actual, requiriendo lógica decodificadora completa en un solo bloque.

   Para arquitecturas pipeline, la complejidad aumenta significativamente debido a la necesidad de implementar:

   - **Registros interetapa (IF/ID, ID/EX, EX/MEM, MEM/WB):** diseño, manejo de enable/flush signals, propagación correcta de señales de control a través de las etapas (Harris & Harris, 2021)
   - **Unidad de detección de hazards:** lógica que identifica dependencias RAW (Read-After-Write) entre instrucciones en diferentes etapas del pipeline, comparando registros fuente de instrucción en ID con registros destino de instrucciones en EX, MEM y WB. Esta unidad debe operar en el camino crítico y puede afectar la frecuencia máxima alcanzable (Patterson & Hennessy, 2017).
   - Unidad de forwarding: multiplexores adicionales en la entrada de la ALU que permiten redireccionar datos desde etapas MEM/WB directamente a EX, evitando stalls innecesarios. Esto requiere 4-8 paths de forwarding con lógica de selección compleja (Mnaaoui et al., 2018).
   - Lógica de control de stalling: capacidad de detener selectivamente etapas del pipeline insertando burbujas (NOPs) cuando hazards inevitables (load-use) son detectados, manteniendo coherencia del estado del procesador (Patterson & Hennessy, 2017).
   - Manejo de branches: detección temprana de saltos en etapa ID o EX, cálculo de dirección de salto, lógica de flush del pipeline cuando branch es taken (descartando

instrucciones incorrectamente fetched), y opcionalmente, branch prediction con tablas de historia y actualización dinámica (Patterson & Hennessy, 2017).

2. Para que una comparación entre arquitecturas monociclo y pipeline sea científicamente válida, es imperativo garantizar que ambas implementaciones ejecutan exactamente la misma ISA con corrección funcional idéntica. Esto requiere verificación exhaustiva que consume tiempo y recursos significativos.

   La verificación de conformidad funcional en procesadores RISC-V suele basarse en suites oficiales como riscv-tests y riscv-compliance, que proporcionan colecciones de pruebas dirigidas para distintos subconjuntos de la ISA. En particular, los conjuntos de pruebas para RV32I y RV32IM incluyen varias decenas de programas enfocados por instrucción y por categoría (por ejemplo, rv32ui para instrucciones enteras básicas y rv32um para operaciones de multiplicación y división), cubriendo instrucciones individuales, casos límite como overflow aritmético o división por cero, y patrones de dependencias entre instrucciones consecutivas. Ejecutar y validar estas suites para cada microarquitectura requiere instrumentar el procesador para cargar y ejecutar cada test, recoger el resultado de salida esperado y confirmar automáticamente la ausencia de fallos, lo que convierte la verificación en una fase sustancial del esfuerzo total de desarrollo.

   - Infraestructura de testbench: código SystemVerilog/Verilog que carga programas de prueba en la memoria de instrucciones, ejecuta el procesador, captura el estado final (registros y memoria) y lo compara contra valores esperados para determinar si el comportamiento es correcto.
   - Debugging iterativo: cuando una prueba falla, es necesario identificar la causa raíz en el código RTL, corregir, volver a sintetizar si corresponde y reejecutar los tests. En microarquitecturas pipeline, los errores relacionados con hazards de datos o de control suelen ser especialmente difíciles de detectar porque dependen de patrones de dependencias entre instrucciones y de condiciones de carrera temporales, por lo que pueden manifestarse solo bajo secuencias específicas de ejecución. Este tipo de bugs se considera en la práctica uno de los desafíos más complejos en el ciclo de verificación de procesadores pipeline.
   - Validación de equivalencia: después de pasar suites como riscv-tests de manera individual, es necesario comprobar que las dos microarquitecturas den resultados exactamente coincidentes (bit a bit) para programas más complejos, no solo que ambas sean funcionalmente correctas en forma aislada. Esto implica ejecutar las mismas cargas de trabajo sobre ambos diseños, capturar los estados finales relevantes (registros, memoria) y compararlos de forma sistemática. En la práctica,

diferentes reportes de la industria y de la comunidad RISC-V coinciden en que la verificación funcional y la depuración de procesadores consumen una fracción muy significativa del esfuerzo total de desarrollo debido a la necesidad de combinar pruebas dirigidas, suites de conformidad, generación de tests aleatorios y, cada vez más, herramientas de verificación formal como riscv-formal. Dado que gran parte de este esfuerzo debe repetirse para cada variante de microarquitectura, implementar y verificar simultáneamente una versión monociclo y una versión pipeline tiende a incrementar sustancialmente la carga total de verificación.

3. La comunidad de investigación en arquitectura de computadores implementa diseños RISC-V en una diversidad de plataformas FPGA de diferentes fabricantes y generaciones tecnológicas, cada una con características arquitecturales distintivas que afectan profundamente el rendimiento sintetizado.

   Diferencias entre familias FPGA:

   - Arquitectura de LUTs: dispositivos Xilinx de la familia 7-series utilizan bloques lógicos basados en LUTs de 6 entradas integradas en CLBs, mientras que dispositivos Intel/Altera como Cyclone V emplean módulos lógicos adaptativos (ALMs) que agrupan dos LUTs fracturables con hasta 8 entradas compartidas, además de lógica adicional de suma, multiplexores y flip-flops. Estas diferencias en la organización interna influyen en cómo las funciones lógicas complejas se mapean a recursos físicos y, en consecuencia, en el retardo de los caminos críticos y la utilización de recursos reportada por las herramientas de síntesis (Percey, 2007).
   - Block RAM: La arquitectura interna de memoria también difiere significativamente; los dispositivos Cyclone V de Intel proporcionan bloques M10K de 10 Kbits y MLABs más pequeños, mientras que las familias equivalentes de Xilinx ofrecen bloques BRAM configurables de 18 o 36 Kbits. Estas diferencias en capacidad nativa, latencia de acceso y modos de operación afectan directamente la eficiencia con la que se implementan componentes críticos del procesador, tales como las memorias de instrucción y datos, o el banco de registros (Intel Corporation, 2016; Xilinx, 2020)
   - DSP blocks: Los dispositivos Cyclone V de Intel incorporan bloques DSP de precisión variable que permiten implementar multiplicadores de 18x18, 27x27 y configuraciones derivadas, mientras que las FPGAs Xilinx 7-series integran slices DSP48E1 con un multiplicador típico de 25x18 bits junto con pre-adders y acumuladores de 48 bits. Estas diferencias en la arquitectura interna de los bloques

DSP y en su modo de integración con la lógica de la FPGA influyen directamente en cómo se implementa la extensión M de RISC-V, pudiendo favorecer una u otra familia según el tamaño de operandos, el grado de pipeline interno y el estilo de datapath empleado (Intel Corporation, 2018; Xilinx, 2014).

- Herramientas de síntesis: Las cadenas de herramientas empleadas utilizan algoritmos de optimización, estrategias de mapeo tecnológico, ubicación y enrutamiento y análisis de temporización estática distintos. Estas variaciones metodológicas en el flujo CAD implican que un mismo código RTL sintetizado para FPGAs equivalentes en capacidad lógica puede generar frecuencias máximas y reportes de utilización de recursos que no son directamente comparables, añadiendo una capa de incertidumbre al comparar resultados de diferentes plataformas (Heinz et al., 2019; Murray et al., 2020).

4. A diferencia de los procesadores comerciales, donde benchmarks como SPEC CPU, CoreMark y Dhrystone cuentan con protocolos de ejecución, reporte y validación estrictamente definidos por consorcios y organizaciones como EEMBC y SPEC (EEMBC, 2009), la comunidad de implementaciones RISC-V en FPGA carece de estándares ampliamente adoptados para la medición y el reporte de rendimiento.

   Problemas específicos identificados en la literatura:

   - Variabilidad en configuración de benchmarks: Diferentes estudios que evalúan procesadores RISC-V en FPGA ejecutan CoreMark con configuraciones heterogéneas, por ejemplo, variando el tiempo total de ejecución, las opciones del compilador o incluso la forma de reportar el resultado, lo que dificulta la comparación directa entre implementaciones. Heinz et al. (2019) muestran que parámetros como la configuración del compilador y de la plataforma tienen un impacto notable en los resultados de CoreMark para distintos softcores RISC-V, reforzando la necesidad de protocolos más homogéneos de medición.
   - Falta de normalización: En muchos trabajos sobre implementaciones RISC-V en FPGA se reporta el resultado de CoreMark como un valor absoluto (por ejemplo, "2500 CoreMarks") sin normalizar por la frecuencia de operación, lo que hace inválida la comparación directa entre diseños que corren a distintas frecuencias. La métrica recomendada en la documentación de CoreMark es precisamente CoreMark/MHz, que se obtiene dividiendo el score por la frecuencia de reloj y permite comparar de forma más justa la eficiencia arquitectural entre procesadores con pipelines o tecnologías distintas, al reducir el efecto de la frecuencia de reloj sobre el resultado (Analog Devices, 2023; Arm Limited, 2022; EEMBC, 2009).

- Ausencia de análisis estadístico: En la mayoría de los trabajos sobre implementaciones de procesadores RISC-V en FPGA los benchmarks se ejecutan una sola vez y se reporta un único valor de rendimiento sin incluir múltiples ejecuciones ni estadísticas descriptivas como media y desviación estándar. Este enfoque ignora la variabilidad introducida por factores como diferencias entre compilaciones, condiciones de temperatura en la FPGA o pequeñas variaciones en el flujo de síntesis y place-and-route, lo que reduce la confiabilidad de las conclusiones extraídas a partir de un único dato puntual.

2.5. Definición del problema

Aunque existen múltiples implementaciones RISC-V en FPGA, los estudios previos no ofrecen comparaciones directas ni homogéneas entre una arquitectura monociclo y una arquitectura pipeline de cinco etapas para un procesador RV32IM en la misma plataforma FPGA. Esta falta de evidencia comparable dificulta establecer con claridad si la microarquitectura pipeline realmente proporciona ventajas de rendimiento frente a un diseño monociclo equivalente bajo condiciones controladas.

2.6. Consecuencias

La ausencia de protocolos estandarizados de comparación limita la posibilidad de integrar resultados provenientes de distintos estudios. Cada trabajo aporta datos parciales que no pueden relacionarse adecuadamente con implementaciones similares, lo que impide construir conclusiones sólidas sobre los trade-offs reales entre monociclo y pipeline. Cuando las mediciones de rendimiento, utilización de recursos o CPI se realizan con metodologías diferentes, los resultados no son replicables ni comparables por otros investigadores. Esto afecta la confiabilidad científica y dificulta la validación independiente de hallazgos.

3. Justificación

3.1. Justificación científica y tecnológica

La arquitectura RISC-V se ha consolidado como un referente en investigación y desarrollo debido a que es abierta, modular y extensible, lo que ha motivado numerosas implementaciones en FPGA orientadas a explorar rendimiento y eficiencia en sistemas embebidos. En este contexto, realizar una comparación experimental controlada entre una microarquitectura RV32IM monociclo y una microarquitectura RV32IM segmentada en cinco etapas, ambas implementadas sobre la misma FPGA Cyclone V y bajo un protocolo homogéneo de síntesis, verificación y medición, permite contrastar las predicciones teóricas sobre throughput y frecuencia con resultados obtenidos en hardware real. Esto aporta evidencia objetiva sobre los trade-offs de diseño entre simplicidad microarquitectural y complejidad del pipeline, contribuyendo a reducir el vacío existente en la literatura respecto a comparaciones directas para un mismo procesador y plataforma.

3.2. Justificación educativa y social

Desarrollar un procesador RISC-V RV32IM completo en la tarjeta DE1-SoC implica abordar restricciones reales de recursos lógicos, bloques de memoria, bloques DSP y márgenes de temporización propios de la FPGA Cyclone V, así como las limitaciones de las herramientas de síntesis y place-and-route. El proyecto permite cuantificar, en un entorno representativo de laboratorio, el impacto de pasar de una arquitectura monociclo a una segmentada sobre la frecuencia máxima alcanzable, el CPI efectivo, el throughput y la utilización de recursos, aportando datos sobre la viabilidad y el desempeño real de procesadores RV32IM en este tipo de plataformas. Estos resultados pueden servir como referencia práctica para diseñadores e investigadores que deban decidir entre implementar procesadores monociclo o pipeline en FPGAs de capacidad similar.

3.3. Justificación Técnica

Aunque el objetivo principal del estudio es técnico y se centra en la comparación de rendimiento, el procesador resultante y la consola de visualización de estados internos constituyen un recurso potencialmente valioso para la formación en arquitectura de computadores. La posibilidad de observar el flujo de instrucciones a través de las etapas del pipeline, las señales de control y la resolución de hazards ofrece una herramienta de apoyo para la docencia y la experimentación en cursos de sistemas digitales y procesadores, facilitando la conexión entre la teoría y el comportamiento observable en hardware reconfigurable

4. Objetivos

4.1. Objetivo General

Desarrollar en FPGA una microarquitectura RISC-V RV32IM segmentada en cinco etapas, acompañada de una pantalla que permita la visualización paso a paso de los estados internos del pipeline.

4.2. Objetivos Específicos

1. Definir la microarquitectura pipeline de cinco etapas (IF, ID, EX, MEM, WB) y su especificación en SystemVerilog con módulos claramente separados.

2. Desarrollar un emulador que muestre en tiempo real los registros interetapa, señales de control, PC y traza de instrucciones, ejecución continua orientadas al uso docente.

3. Construir un conjunto de testbenches y un banco de pruebas que verifiquen funcionalmente el conjunto RV32IM, incluyendo casos de hazards, saltos y operaciones M.

5. Metodología

5.1. Marco metodológico

5.1.1. Descripción de tratamientos

En este estudio se comparan experimentalmente dos variantes de un mismo procesador RISC-V:

- **Tratamiento 1**: microarquitectura RV32IM monociclo, que ejecuta cada instrucción en un único ciclo de reloj.
- **Tratamiento 2**: microarquitectura RV32IM segmentada en cinco etapas (IF, ID, EX, MEM, WB), con resolución explícita de *hazards* de datos y control.

Ambas versiones se diseñan en SystemVerilog, se sintetizan en la misma FPGA Intel Cyclone V DE1-SoC utilizando Intel Quartus Prime, y se someten a los mismos protocolos de verificación funcional que será la suite riscv-tests y evaluación de rendimiento que serán benchmarks CoreMark y programas de prueba representativos.

5.1.2. Unidad experimental

En este estudio, la unidad experimental se define como cada instancia sintetizada del procesador ejecutando un caso de prueba específico bajo condiciones controladas. Es decir, no se considera como unidad experimental la FPGA física de forma abstracta ni el diseño RTL en general, sino la configuración concreta del hardware sintetizado correspondiente a una de las microarquitecturas cuando ejecuta un test o benchmark determinado.

Cada unidad experimental se evalúa de manera independiente durante la ejecución de cada caso de prueba. El comportamiento se registra mediante instrumentación embebida en el propio diseño del procesador y en la lógica auxiliar de medición, que incorpora contadores de ciclos, registros para el Program Counter, señales asociadas a la resolución de hazards y estados de los registros inter-etapa del pipeline.

5.1.3. Número de unidades y respaldo estadístico

Para cada combinación de tratamiento y programa de prueba se ejecutará un mínimo de cinco (5) réplicas independientes. En cada réplica se realizará una nueva síntesis e implementación del diseño en Quartus Prime, seguida de la ejecución del mismo conjunto de pruebas sobre la FPGA, con el fin de capturar posibles variaciones en la frecuencia máxima derivadas del flujo de síntesis y *place-and-route*.

Este tamaño muestral busca un equilibrio entre rigor experimental y viabilidad práctica: cinco ejecuciones permiten estimar estadísticamente la media y una desviación estándar representativa de F_{max} y de las métricas de rendimiento asociadas, sin que el costo computacional de repetir síntesis e implementaciones se vuelva prohibitivo. La elección de cinco réplicas es consistente con prácticas comunes en evaluación de implementaciones hardware y benchmarking de sistemas embebidos, donde la variabilidad proviene principalmente de las heurísticas de las herramientas de síntesis y no de ruido estocástico en la ejecución, y donde se suelen realizar varias corridas para reportar valores promedio y dispersión básica de las métricas.

5.1.4. Diseño experimental y asignación

Se adopta un diseño experimental controlado de medidas repetidas, en el cual cada programa de prueba se ejecuta sobre ambas microarquitecturas bajo condiciones lo más homogéneas posible: misma frecuencia de reloj configurada en tiempo de síntesis, mismos archivos de entrada, mismas opciones de compilación y captura de salidas mediante scripts automatizados que registran las métricas de interés.

En este contexto no se recurre a una asignación aleatoria de tratamientos en el sentido clásico, dado que:

- Los tratamientos corresponden a configuraciones deterministas de hardware sintetizado.
- Todas las ejecuciones se realizan sobre la misma plataforma FPGA bajo condiciones controladas.
- El orden en que se ejecutan los distintos testbenches sobre cada arquitectura no introduce efectos de arrastre, pues cada ejecución comienza desde un estado inicial bien definido y las corridas son independientes entre sí.

5.1.5. Procedimiento y registro de información

El estudio experimental compara dos microarquitecturas: un procesador RISC-V RV32IM monociclo y un modelo *pipeline* de cinco etapas. Para cada tratamiento, el procedimiento inicia con el flujo de síntesis completo en Intel Quartus Prime a partir del cual se extraen la frecuencia máxima estimada, el reporte de utilización de recursos y se genera el archivo de configuración.

Posteriormente, ambas arquitecturas se evalúan mediante una batería de pruebas automatizadas controladas por el entorno de verificación cocotb. Este entorno se encarga de cargar y ejecutar programas representativos sobre el diseño RTL de cada arquitectura, monitorizando

y registrando en tiempo de simulación señales internas clave como el número total de ciclos de reloj, la cantidad de instrucciones retiradas, el CPI, el IPC y los eventos de hazards en el caso del pipeline.

Para garantizar la fiabilidad del estudio, cocotb compara automáticamente los estados finales contra un modelo de referencia en Python, verificando que la ejecución haya sido funcionalmente correcta antes de proceder al análisis de rendimiento. Cada programa se ejecuta en cinco réplicas por tratamiento para minimizar variaciones y asegurar la consistencia experimental.

5.1.6. Variables de respuesta e independientes

En esta investigación se registran diversas métricas cuantitativas para evaluar cómo cambia el rendimiento del procesador al emplear una arquitectura monociclo frente a una arquitectura pipelinede cinco etapas. Estas métricas se agrupan en variables de respuesta, variables independientes y variables de control.

Las **variables de respuesta** son las que permiten medir el desempeño efectivo del procesador en cada tratamiento. Incluyen:

- El número medio de ciclos por instrucción (CPI).
- El número medio de instrucciones retiradas por ciclo (IPC).
- La capacidad de procesamiento por segundo (throughput), derivada de la combinación de F_{max} y CPI.
- La frecuencia máxima alcanzable sin violaciones de temporización (F_{max}) reportada por el Timing Analyzer.
- La utilización de recursos de la FPGA.
- La latencia total requerida para ejecutar cada programa de prueba.

Estas mediciones se obtienen a partir de contadores internos integrados en el procesador, de los reportes de síntesis y temporización generados por Intel Quartus Prime y de la instrumentación de verificación y registro implementada con `cocotb`.

La **variable independiente principal** es el tipo de microarquitectura implementada. Adicionalmente, los distintos programas de prueba utilizados se consideran como un segundo factor experimental, ya que permiten observar el comportamiento de cada arquitectura bajo diferentes patrones de carga.

Las **variables de control** son aquellos parámetros que se mantienen constantes para garantizar una comparación justa entre tratamientos. Entre ellas se incluyen: el conjunto de

instrucciones, la plataforma hardware, la versión de las herramientas de síntesis y compilación utilizadas, la configuración del entorno de compilación, el conjunto de instrucciones cargado en memoria para cada prueba y la frecuencia de reloj de entrada configurada para la síntesis y la ejecución.

5.1.7. Criterios para suspensión o replanteamiento del proyecto

Se establecen criterios explícitos para identificar situaciones en las que la continuación del proyecto, en los términos originalmente definidos, dejaría de ser viable o metodológicamente válida. En tales casos, se considerará la suspensión o el replanteamiento del alcance:

- **Imposibilidad de síntesis en la FPGA objetivo:** si alguno de los tratamientos no puede ser implementado exitosamente en la tarjeta DE1-SoC debido a que la utilización de recursos supera la capacidad disponible de LUTs, FFs, BRAM o bloques DSP del dispositivo, aun después de aplicar optimizaciones razonables de diseño, no será posible realizar la comparación propuesta y se requerirá redefinir el alcance del proyecto.

- **Corrección funcional no alcanzada:** si, tras un número significativo de iteraciones de depuración, los testbenches basados en cocotb no logran confirmar la corrección funcional del procesador el proyecto no podrá avanzar a la fase de evaluación de rendimiento, dado que cualquier comparación carecería de validez.

- **Inestabilidad en las métricas entre réplicas:** si, después de repetir las mediciones, no se consigue reproducir resultados consistentes entre réplicas para un mismo tratamiento y benchmark, por ejemplo, si el coeficiente de variación del CPI entre las cinco réplicas excede un umbral considerado razonable indicando inestabilidad no explicada o posibles errores no deterministas en el diseño o en el proceso de medición, será necesario revisar en profundidad la metodología y, de no resolverse, reconsiderar el alcance del estudio.

- **Restricciones de tiempo y avance mínimo**: si no se logra completar la implementación básica de ambas microarquitecturas, la verificación funcional y al menos una iteración completa de evaluación de rendimiento con un conjunto representativo de unidades experimentales dentro de los primeros cinco meses del proyecto, se deberá evaluar una reducción o ajuste del alcance, priorizando la obtención de resultados parciales válidos antes que mantener un objetivo irrealizable.

5.1.8. Análisis y evaluación de la hipótesis

La evaluación de la hipótesis se basa en el análisis de las variables de respuesta obtenidas para cada tratamiento: CPI, IPC, throughput y frecuencia máxima. Estas variables permiten caracterizar el rendimiento de ambas arquitecturas y constituyen la base para determinar si la microarquitectura pipeline ofrece mejoras cuantificables frente al diseño monociclo.

El análisis comienza con una etapa descriptiva, en la que se resumen las mediciones de cada tratamiento mediante medidas de tendencia central y dispersión para cada benchmark y arquitectura. Esta etapa permite identificar patrones generales, variabilidad entre réplicas y posibles valores atípicos que pudieran influir en la interpretación de los resultados.

Dado el tamaño muestral reducido por combinación de tratamiento y caso de prueba, el énfasis se pone en la comparación de medias y en el cálculo de métricas derivadas, más que en la aplicación extensiva de pruebas de significancia. No obstante, cuando resulte pertinente, se podrán emplear pruebas estadísticas sencillas como apoyo a la interpretación, siempre dejando claro el tamaño de la muestra y las limitaciones asociadas.

Además de la comparación directa de medias, se calcula el speedup del pipeline respecto al monociclo en términos de throughput, así como indicadores de eficiencia relativa que relacionan el rendimiento obtenido con los recursos de FPGA utilizados y el overhead de recursos del pipeline frente al diseño monociclo. Estas métricas permiten evaluar el trade-off entre rendimiento y costo en hardware y aportan una base más completa para valorar si la mejora observada en throughput y F_{max} justifica la complejidad adicional de la arquitectura pipeline.

5.1.9. Evaluación de la hipótesis de trabajo

Hipótesis de investigación (H_1). La implementación de una microarquitectura RISC-V RV32IM segmentada en cinco etapas sobre FPGA ofrece un rendimiento superior, en términos de throughput, respecto a un procesador monociclo equivalente, bajo condiciones controladas de síntesis, verificación y ejecución, manteniendo en ambos casos la corrección funcional.

Hipótesis nula (H_0). No se observa una mejora de rendimiento en throughput a favor de la arquitectura segmentada frente a la monociclo; es decir, la diferencia de throughput entre ambas microarquitecturas es nula o no favorable al pipeline dentro del margen de variación experimental.

5.2. Hipótesis

La implementación de una microarquitectura RISC-V RV32IM segmentada en cinco etapas sobre FPGA mejora el rendimiento, en términos de throughput y frecuencia, frente a un procesador monociclo equivalente, manteniendo la corrección funcional.

5.3. Actividades

Objetivo específico 1. Definir e implementar la microarquitectura pipeline de cinco etapas RV32IM en SystemVerilog.

Actividad 1.1. Diseñar la arquitectura del procesador segmentado, detallando la función e interacción de cada una de las cinco etapas (IF, ID, EX, MEM, WB), los registros interetapa y las señales de control clave, en concordancia con la especificación RV32IM.
Entregable: Diagrama de bloques funcional y documento descriptivo de la microarquitectura.

Actividad 1.2. Codificar en SystemVerilog cada etapa del pipeline y los módulos auxiliares de forma modular, asegurando una separación clara entre etapas y la correcta implementación de la funcionalidad definida en la actividad anterior.
Entregable: Código fuente modular en SystemVerilog para la microarquitectura pipeline RV32IM.

Actividad 1.3. Integrar los módulos del procesador segmentado, sintetizar el diseño completo para la FPGA DE1-SoC y realizar una verificación funcional básica que confirme el correcto funcionamiento conjunto de las etapas.
Entregable: Módulo superior funcional para FPGA y reporte de síntesis inicial con evidencias de operación correcta.

Objetivo específico 2. Desarrollar una visualización básica que permita observar el estado interno del pipeline durante la ejecución del procesador.

Actividad 2.1. Definir y seleccionar las señales internas del pipeline que serán expuestas para visualización, así como el medio de salida más adecuado.
Entregable: Documento breve con un esquema de la visualización propuesta y la lista de señales seleccionadas.

Actividad 2.2. Implementar en SystemVerilog el módulo de visualización elegido e integrarlo con el procesador para mostrar en tiempo (casi) real los estados internos seleccionados durante la ejecución de programas de prueba, asegurando que la lógica de visualización no altere la funcionalidad del procesador.
Entregable: Código funcional del módulo de visualización y evidencia de una sesión real demostrando su uso.

Objetivo específico 3. Verificar la correcta operación del procesador RV32IM mediante

testbenches automatizados desarrollados con cocotb, obteniendo además métricas básicas de desempeño.

Actividad 3.1. Elaborar un plan mínimo de verificación en cocotb que incluya instrucciones representativas, casos de hazards y al menos una operación por cada tipo de instrucción del conjunto RV32IM.

Entregable: Lista documentada de casos de prueba, con su clasificación y cobertura sobre el conjunto de instrucciones.

Actividad 3.2. Desarrollar y ejecutar testbenches automáticos con cocotb para verificar la funcionalidad del procesador, comparar los resultados con un modelo de referencia y recopilar métricas como CPI, throughput y correctitud funcional frente a los resultados esperados.

Entregable: Scripts de cocotb, registros de los resultados principales y un breve reporte de la verificación realizada.

6. Aportes

- **Repositorio abierto del procesador RV32IM segmentado**
  Código fuente completo en SystemVerilog de un procesador RISC-V RV32IM con microarquitectura segmentada en cinco etapas, publicado en un repositorio público y documentado para su reutilización en contextos académicos y de investigación (docencia de arquitectura, experimentos de rendimiento, extensiones de ISA, etc.).

- **Módulo de visualización del pipeline sobre FPGA**
  Módulo de visualización integrado en la FPGA que permite observar, en tiempo casi real, el comportamiento interno del pipeline (PC, registros interetapa, señales de control y detección de hazards), con potencial uso como herramienta docente en cursos de arquitectura de computadores y diseño de procesadores.

- **Conjunto de testbenches automatizados con cocotb**
  Conjunto de testbenches automatizados basados en cocotb para la verificación funcional de procesadores RISC-V, que cubren instrucciones RV32IM, casos de hazards y generación de métricas de rendimiento como CPI y throughput, sirviendo como punto de partida reutilizable para otros diseños de núcleos RISC-V.

7. Marco referencial

7.1. Marco teorico

7.1.1. Arquitectura de Computadores y Pipeline

El diseño de arquitecturas modernas se fundamenta en principios cuantitativos que buscan optimizar el rendimiento. Según Patterson y Hennessy (2017), el pipelining es una técnica de implementación clave donde múltiples instrucciones se superponen en ejecución, funcionando de manera análoga a una línea de ensamblaje industrial. En su obra, se describen detalladamente las cinco etapas clásicas del pipeline RISC que se implementarán en este proyecto: *Instruction Fetch* (IF), *Instruction Decode* (ID), *Execute* (EX), *Memory Access* (MEM) y *Write Back* (WB).

Patterson y Hennessy (2017) desarrollan las ecuaciones fundamentales de rendimiento que relacionan el tiempo de ejecución con el número de instrucciones, los ciclos por instrucción (CPI) y el tiempo de ciclo de reloj. Un procesador monociclo mantiene un CPI = 1 pero requiere un ciclo de reloj extenso (determinado por la instrucción más lenta); en contraste, un procesador segmentado busca alcanzar un CPI cercano a 1 permitiendo ciclos de reloj significativamente más cortos.

Sin embargo, el rendimiento del pipeline se ve afectado por tres tipos de *hazards* o riesgos (Patterson & Hennessy, 2017):

1. **Hazards estructurales:** Causados por conflictos de recursos en el hardware.

2. **Hazards de datos:** Resultantes de dependencias entre instrucciones que requieren resultados aún no escritos.

3. **Hazards de control:** Introducidos por instrucciones de salto que alteran el flujo secuencial.

Para mitigar estos efectos, se presentan técnicas de resolución como el forwarding, el stalling y la predicción de saltos. Esta obra es fundamental para el proyecto, ya que proporciona la base teórica para comprender la mejora del throughput y las métricas de rendimiento en la microarquitectura RV32IM segmentada.

7.1.2. Diseño Digital y Arquitectura RISC-V

Para la implementación práctica en hardware, la obra de Harris y Harris (2021) proporciona una transición integral desde los fundamentos del diseño digital hasta la construcción de arquitecturas complejas. Los autores utilizan el conjunto de instrucciones RISC-V como caso

de estudio unificador, cubriendo desde puertas lógicas básicas hasta el diseño de procesadores con pipeline.

El texto detalla la especificación RV32I y sus extensiones estándar, con especial énfasis en la extensión M para operaciones de multiplicación y división por hardware. Se describen minuciosamente los formatos de instrucción (R, I, S, B, U, J), incluyendo diagramas precisos sobre la codificación de campos como *opcode*, *funct3*, *funct7*, registros fuente/destino e inmediatos (Harris & Harris, 2021).

Harris y Harris (2021) presentan dos implementaciones de referencia cruciales para este proyecto:

- **Procesador monociclo:** Una implementación simple donde cada instrucción se ejecuta en un único ciclo de reloj.
- **Procesador segmentado:** Un diseño de cinco etapas que requiere la implementación de registros interetapa para almacenar señales de control y datos entre fases consecutivas.

Para asegurar la corrección funcional del diseño segmentado, los autores detallan la lógica de detección de hazards y la unidad de forwarding. Además, el enfoque en el "diseño modular" mediante SystemVerilog que proponen Harris y Harris resulta fundamental para este anteproyecto, ya que facilita la verificación y el debugging de cada módulo independiente al ser implementados en la FPGA DE1-SoC.

7.1.3. Especificación oficial RISC-V

Waterman y Asanović (2019) presentan la especificación oficial del conjunto de instrucciones RISC-V a nivel de usuario, definiendo formalmente el conjunto base RV32I y las extensiones estándar. La obra describe para cada instrucción su semántica precisa, la codificación binaria y el comportamiento observable, estableciendo la referencia de corrección funcional para cualquier implementación compatible.

En la especificación se introduce un conjunto base mínimo (RV32I), con 40 instrucciones suficientes para servir como objetivo de compilador, que puede ampliarse mediante extensiones opcionales identificadas por letras, como la extensión "M" para multiplicación y división enteras. La extensión estándar "M" incorpora ocho instrucciones aritméticas (`MUL`, `MULH`, `MULHU`, `MULHSU`, `DIV`, `DIVU`, `REM` y `REMU`), permitiendo que una implementación RV32I se extienda a RV32IM manteniendo la compatibilidad con la semántica definida por la ISA.[web:233][web:241]

Esta especificación es fundamental para el proyecto porque fija el comportamiento observable del procesador RV32IM y sirve como contrato formal tanto para el diseño de la

microarquitectura como para la construcción de testbenches de verificación que comparan resultados contra un modelo ISA.

7.1.4. Métricas de rendimiento en arquitectura de computadores

Ramos-Hernandez y Tokhi (2003) utilizan la ecuación clásica de rendimiento de arquitectura de computadores, donde el tiempo de CPU se expresa como el producto entre el número de instrucciones ejecutadas, el CPI medio y el tiempo de ciclo de reloj, descomponiendo así el rendimiento en estos tres factores. Este enfoque permite analizar por separado el impacto de optimizaciones a nivel de compilador, de microarquitectura y de hardware.

Por su parte, Yi et al. (2020) reevalúan el uso de CPI como métrica de rendimiento en tiempo de ejecución y muestran que puede presentar sesgos debidos a la frecuencia de núcleo y a la intensidad de carga, de modo que un CPI aparentemente mejor no siempre se traduce en un menor tiempo de ejecución observado. Para mitigar estas limitaciones, proponen analizar desgloses de CPI en componentes asociados a distintos eventos microarquitecturales y complementarlo con métricas alternativas orientadas a identificar cuellos de botella específicos.

7.1.5. Síntesis en FPGA y análisis de timing

Chu (2011) presenta una metodología práctica de diseño para FPGAs utilizando SystemVerilog como lenguaje de descripción de hardware, mostrando cómo las descripciones a nivel RTL se traducen en LUTs, flip-flops, memorias embebidas y otros recursos especializados del dispositivo. El texto enfatiza el uso de buenas prácticas de codificación y estructuras modulares para obtener implementaciones eficientes y portables entre distintas familias de FPGAs.

En el contexto del análisis de timing, Chu destaca que la inserción de registros intermedios para segmentar caminos combinacionales largos es una técnica clave para aumentar la frecuencia máxima de operación, al reducir la profundidad lógica entre registros consecutivos. Este enfoque de diseño pipeline permite alcanzar mayores F_{max}, pero a costa de incrementar el número total de registros y, por tanto, el uso de recursos y la latencia en ciclos de reloj, lo que introduce un compromiso entre rendimiento, área y latencia que debe considerarse en el diseño sobre FPGA.

7.2. Estado del arte

7.2.1. Introducción

La arquitectura RISC-V ha experimentado un crecimiento en la última década, consolidándose como una alternativa abierta y flexible para la investigación y el desarrollo de procesadores, gracias a su especificación libre y extensible.(Patterson & Hennessy, 2017; Waterman & Asanović, 2019) Esta apertura ha favorecido la proliferación de núcleos RISC-V de propósito general y embebido, muchos de ellos orientados específicamente a su implementación como *softcores* sobre FPGA en contextos académicos e industriales.(Chu, 2011; Heinz et al., 2019)

La posibilidad de sintetizar microarquitecturas RISC-V en FPGAs ha democratizado el acceso a plataformas de prototipado rápido, permitiendo a universidades y grupos de investigación desarrollar y evaluar sus propios diseños sin la barrera de licencias propietarias.(Chu, 2011; Nguyen & Le, 2024) Sin embargo, los estudios existentes suelen centrarse en una única microarquitectura o bien en comparaciones entre diferentes núcleos RISC-V ya existentes, más que en comparaciones controladas entre variantes del mismo procesador sobre una misma FPGA.(Heinz et al., 2019; Liu et al., 2021)

En particular, la literatura científica revisada no ofrece todavía un análisis que compare, bajo condiciones de síntesis y medición homogéneas, una implementación monociclo y una implementación segmentada de un mismo núcleo RV32IM en una plataforma FPGA concreta, reportando sistemáticamente métricas como frecuencia máxima, CPI/IPC y *throughput*.(Heinz et al., 2019; Liu et al., 2021; Patterson & Hennessy, 2017) Este vacío justifica la necesidad de un estudio experimental que contraste directamente ambas microarquitecturas dentro de un marco de evaluación común.

A partir de esta observación, el presente estado del arte organiza y analiza críticamente los trabajos previos en seis ejes temáticos:

1. Implementaciones monociclo de RV32I/RV32IM en FPGA.
2. Implementaciones segmentadas de RV32I/RV32IM en FPGA.
3. Técnicas de resolución de *hazards*.
4. Estudios comparativos multi-arquitectura.
5. Metodologías de verificación funcional.
6. *Benchmarks* y métricas de rendimiento.

Este análisis permite identificar la brecha de conocimiento que el proyecto pretende abordar, situando la comparación entre procesador monociclo y pipeline RV32IM sobre FPGA dentro del contexto actual de investigación en RISC-V.

7.2.2. Implementaciones monociclo de RV32I/RV32IM en FPGA

Las arquitecturas monociclo representan el enfoque más directo para implementar un procesador RISC-V, en el que cada instrucción se completa conceptualmente en un único ciclo de reloj, reutilizando un mismo datapath para todas las operaciones.(Harris & Harris, 2021) Esta simplicidad estructural y de control hace que los procesadores de ciclo único sean especialmente atractivos con fines docentes, al facilitar la comprensión del flujo de datos y de las señales de control en la ruta de datos completa.(Anjana et al., 2025; Harris & Harris, 2021)

Sin embargo, el requisito de que todas las instrucciones respeten el mismo periodo de reloj implica que el tiempo de ciclo debe dimensionarse según el camino crítico más largo, lo que limita la frecuencia máxima alcanzable y, por tanto, el rendimiento global del procesador en tecnologías FPGA modernas.(Chu, 2011; Patterson & Hennessy, 2017) Aunque las implementaciones monociclo de núcleos RV32I/RV32IM sobre FPGA son valiosas como primer paso y como plataforma educativa, suelen considerarse un punto de partida para diseños más avanzados basados en microarquitecturas segmentadas o multiciclo.

7.2.3. Procesador RV32I monociclo en Intel Cyclone V DE1-SoC

Reddy et al. (2024) implementan un procesador RV32I de ciclo único optimizado específicamente para la FPGA Intel Cyclone V presente en la plataforma DE1-SoC, describiendo el núcleo en Verilog y soportando 23 instrucciones del conjunto base RV32I. Tras síntesis y place-and-route con Intel Quartus Prime, el diseño alcanza una frecuencia máxima de aproximadamente 30,35 MHz, con un CPI medio de 1,0 coherente con la naturaleza monociclo de la arquitectura, y un uso cercano a 2 500 elementos lógicos (LUTs), acompañado de un consumo de potencia estimado en torno a 424,45 mW según el Power Analyzer de Quartus. La verificación funcional se realiza mediante simulación con ModelSim, confirmando la ejecución correcta de todas las instrucciones implementadas sin violaciones de timing reportadas en las condiciones de prueba.

Este estudio ofrece un baseline cuantitativo directo para arquitecturas monociclo en la plataforma DE1-SoC, proporcionando puntos de referencia concretos en términos de frecuencia máxima y consumo de potencia que resultan útiles para valorar si una implementación segmentada en la misma FPGA justifica su mayor complejidad mediante mejoras proporcio-

nales en throughput. No obstante, el trabajo se limita al conjunto base RV32I, sin incluir la extensión M de multiplicación y división, y no reporta resultados sobre benchmarks estandarizados como CoreMark o Dhrystone, lo que deja abierta la cuestión de cómo se comportaría el diseño bajo cargas de trabajo más representativas.

7.2.4. Procesador RV32I compacto con verificación rigurosa

Kumar y Singh (2023) presentan una implementación RV32I de ciclo único compacta y eficiente en energía sobre una FPGA Xilinx Spartan-7, poniendo especial énfasis en una metodología de verificación rigurosa basada en la suite oficial riscv-tests. El procesador, descrito en Verilog HDL e incorporando optimizaciones de bajo consumo como clock gating y compartición de recursos, alcanza tras síntesis y place-and-route con Vivado 2021.1 una frecuencia máxima en torno a 45 MHz, utilizando aproximadamente 2 134 LUTs, 1 087 flip-flops y 10 bloques de memoria BRAM.

El análisis de potencia reporta un consumo total cercano a 0,142 W (0,015 W dinámico y 0,127 W estático), lo que sitúa a este diseño como una opción particularmente atractiva para sistemas embebidos con restricciones energéticas. El aporte metodológico central reside en la documentación detallada del protocolo de verificación: la ejecución de la suite completa riscv-tests para RV32I con una tasa de éxito del 100 % establece un estándar de conformidad funcional que resulta directamente replicable en el presente proyecto.

En términos de rendimiento, la frecuencia de 45 MHz sobre Spartan-7 es comparable a los aproximadamente 30 MHz alcanzados por Reddy et al. (2024) en Cyclone V, considerando las diferencias arquitecturales entre familias FPGA (estructura de LUTs y recursos lógicos). No obstante, al igual que otros trabajos centrados en RV32I, este estudio no incorpora la extensión M ni evalúa arquitecturas segmentadas, por lo que no aborda de forma directa el impacto del pipelining sobre el throughput ni sobre el espacio de diseño rendimiento-consumo.

7.2.5. Síntesis comparativa de implementaciones monociclo

Los estudios revisados convergen en métricas consistentes para arquitecturas monociclo RV32I implementadas en FPGAs de capacidad media: las frecuencias máximas típicas se sitúan en el rango de 30–45 MHz, limitadas por el camino crítico combinacional que atraviesa secuencialmente las etapas funcionales necesarias para completar cada instrucción en un único ciclo de reloj.(Kumar & Singh, 2023; Reddy et al., 2024) El CPI medio es igual a 1,0 por definición de la microarquitectura de ciclo único, mientras que el uso de recursos lógicos suele encontrarse en el intervalo de 2 000–2 500 LUTs, lo que representa aproximadamente un 6–8 % de FPGAs con del orden de 32 k LUTs disponibles.(Kumar & Singh, 2023; Reddy

et al., 2024)

Entre las ventajas de estas implementaciones destacan la simplicidad conceptual, la facilidad de depuración y la ausencia de hazards entre instrucciones, al no existir solapamiento temporal en la ejecución.(Harris & Harris, 2021; ScienceDirect Topics, 2024) En contrapartida, la frecuencia de reloj está inherentemente limitada por el camino crítico más largo y el throughput resultante es subóptimo frente a arquitecturas segmentadas o multinivel, especialmente en FPGAs modernas diseñadas para explotar el pipelining profundo.(Chu, 2011; Patterson & Hennessy, 2017)

Una limitación crítica a los trabajos revisados es que ninguno implementa la extensión M de RISC-V, la cual introduce latencias potencialmente multicíclo y condiciona tanto el diseño del datapath como las estrategias de segmentación.(Kumar & Singh, 2023; Reddy et al., 2024; Waterman & Asanović, 2019) Esta ausencia impide evaluar de forma completa el impacto de instrucciones aritméticas complejas en el equilibrio rendimiento-recursos de las arquitecturas monociclo frente a sus contrapartes pipeline, dejando abierta la cuestión que el presente proyecto aborda al considerar explícitamente una microarquitectura RV32IM segmentada.

7.2.6. Implementaciones pipeline de RV32I/RV32IM en FPGA

Las arquitecturas pipeline de cinco etapas constituyen el paradigma dominante en procesadores RISC modernos, dividiendo la ejecución de instrucciones en fases especializadas de búsqueda (IF), decodificación (ID), ejecución (EX), acceso a memoria (MEM) y escritura de resultados (WB) para permitir el solapamiento temporal de múltiples instrucciones. Este enfoque reduce el período de reloj asociado al camino crítico de cada etapa individual y aumenta el throughput del procesador al acercar el CPI efectivo a 1 en condiciones ideales, a costa de introducir hazards que deben tratarse mediante unidades de control adicionales, técnicas de forwarding y mecanismos de parada.

Diversos trabajos recientes han implementado núcleos RV32I con pipeline de cinco etapas sobre FPGA, explotando precisamente esta división en etapas para alcanzar frecuencias de operación superiores a las de sus equivalentes monociclo y para integrar técnicas de optimización como predicción de saltos o mejoras en la unidad de decodificación. En estos diseños, la segmentación del datapath y la incorporación de lógica específica para resolución de hazards de datos y de control son elementos centrales para lograr un buen compromiso entre rendimiento, uso de recursos lógicos y complejidad de verificación en plataformas FPGA.

7.2.7. Pipeline RV32I optimizado con técnicas avanzadas de resolución de hazards

«Performance-Optimised Design of the RISC-V Five-Stage Pipeline Processor» (2024) presentan un procesador RV32I de cinco etapas segmentadas implementado en una FPGA Xilinx Artix-7, cuyo diseño se centra en optimizar el rendimiento mediante mejoras en la unidad de decodificación y en el tratamiento de saltos de control. El núcleo, descrito en Verilog HDL, incorpora mecanismos de resolución de dependencias y optimizaciones en la ruta de control para reducir la degradación de CPI debida a hazards de datos y de control, dentro de la estructura clásica de pipeline IF-ID-EX-MEM-WB.

Los resultados experimentales muestran que, tras aplicar las técnicas propuestas, el procesador alcanza un rendimiento de 3.11 CoreMark/MHz, lo que supone una mejora aproximada del 11.07 % frente a una versión base sin optimizaciones, manteniendo una frecuencia de operación adecuada para la familia Artix-7. El uso de CoreMark como métrica normalizada permite comparar la eficiencia arquitectural del diseño independientemente de la frecuencia absoluta, destacando el impacto de las optimizaciones de decodificación y predicción de saltos sobre el throughput efectivo.

El estudio cuantifica además el coste en recursos de las mejoras introducidas, mostrando un aumento moderado del uso de LUTs asociado a la lógica adicional de control y resolución de hazards, a cambio de un incremento significativo de rendimiento medido por instrucción. Este compromiso área-rendimiento proporciona un punto de referencia relevante para el presente proyecto, al ilustrar cómo técnicas de resolución de dependencias y optimización del flujo de instrucciones pueden traducirse en mejoras medibles de CoreMark/MHz en procesadores RISC-V pipeline sobre FPGA.

7.2.8. Catálogo y evaluación in-hardware de cores RISC-V open-source

Heinz et al. (2019) realizan una evaluación in-hardware de un conjunto de softcores RISC-V de código abierto sobre FPGAs Xilinx Artix-7 y Kintex-7, abarcando tanto núcleos minimalistas como diseños de mayor rendimiento. Su metodología incluye síntesis, place-and-route y ejecución directa del benchmark CoreMark en FPGA, repitiendo el flujo de implementación física cinco veces por núcleo para cuantificar la variabilidad de F_{max} debida a las herramientas de síntesis.

Los resultados ponen de manifiesto un espectro amplio de compromisos entre recursos y rendimiento: núcleos muy compactos como PicoRV32 priorizan el ahorro de LUTs y FFs a costa de un menor rendimiento en CoreMark/MHz, mientras que diseños más complejos alcanzan valores de CoreMark/MHz significativamente superiores pero con un coste de área

que puede llegar a decenas de miles de LUTs, difícilmente asumible en FPGAs educativas de gama media. Entre ambos extremos, núcleos de clase microcontrolador como Ibex ofrecen configuraciones balanceadas, con rendimientos en torno a 2–3 CoreMark/MHz y un uso moderado de recursos lógicos, lo que los hace atractivos para aplicaciones embebidas y docencia.

El estudio cuantifica asimismo la variabilidad de síntesis, observando desviaciones típicas del orden del 3 % en F_{max} entre distintas corridas del flujo EDA, lo que subraya la necesidad de reportar promedios o rangos en lugar de valores puntuales aislados. Considerando que la FPGA Intel Cyclone V de la placa DE1-SoC dispone de un orden de magnitud similar de recursos lógicos que las Artix-7 utilizadas en el estudio, los resultados sugieren que diseños balanceados del tipo Ibex son objetivos alcanzables para un núcleo RV32IM segmentado en este proyecto, con un presupuesto razonable de 5 000–10 000 LUTs para el procesador.(Heinz et al., 2019; Nguyen & Le, 2024)

7.2.9. Síntesis comparativa de implementaciones pipeline

Los trabajos analizados sobre núcleos RV32I de cinco etapas segmentadas en FPGA reportan frecuencias máximas típicas en el rango de 80–100 MHz, aproximadamente entre 2 y 3 veces superiores a las alcanzadas por implementaciones monociclo comparables en las mismas familias de dispositivos.(«Performance-Optimised Design of the RISC-V Five-Stage Pipeline Processor», 2024) El CPI efectivo se sitúa usualmente entre 1.08 y 1.52, reflejando la penalización introducida por los hazards de datos y de control respecto al valor ideal de CPI igual a 1.(«Performance-Optimised Design of the RISC-V Five-Stage Pipeline Processor», 2024)

En términos de recursos lógicos, estos diseños emplean típicamente entre 4 000 y 6 000 LUTs, lo que supone un incremento del orden del 50–100 % frente a procesadores RV32I monociclo de referencia, debido a la lógica adicional de registros interetapa, detección de dependencias y control del pipeline.(«Performance-Optimised Design of the RISC-V Five-Stage Pipeline Processor», 2024) La eficiencia arquitectural medida mediante CoreMark/MHz se encuentra en el rango aproximado de 2.8–3.1 y, combinada con las frecuencias de operación anteriores y los CPI efectivos reportados, da lugar a throughputs en torno a 50–70 MIPS.(«Performance-Optimised Design of the RISC-V Five-Stage Pipeline Processor», 2024)

En conjunto, estos resultados cuantifican un compromiso claro: las microarquitecturas segmentadas incrementan el uso de recursos entre un 50 y un 100 %, pero proporcionan un throughput 2–3 veces superior y una mejora de alrededor del 10–30 % en CoreMark/MHz frente a diseños de complejidad comparable situados en la parte alta del espectro de núcleos monociclo descritos en catálogos de cores RISC-V open-source.(Heinz et al., 2019; «Performance-

Optimised Design of the RISC-V Five-Stage Pipeline Processor», 2024) No obstante, la mayoría de los estudios se centran en evaluar únicamente la variante pipeline o en comparar varios cores distintos, sin disponer de un procesador monociclo y uno segmentado del mismo diseño evaluados en la misma FPGA y bajo un protocolo de medición homogéneo, lo que dificulta valorar de forma controlada si el overhead en recursos y complejidad se justifica plenamente.(Heinz et al., 2019)

7.2.10. Técnicas de resolución de hazards en arquitecturas pipeline

Los hazards representan situaciones que impiden el flujo ideal de instrucciones en un pipeline, incrementando el CPI efectivo y degradando el rendimiento global del procesador al introducir ciclos de espera adicionales.(Patterson & Hennessy, 2017) En la literatura clásica de arquitectura de computadores se distinguen tres categorías principales: hazards de datos, originados por dependencias entre instrucciones; hazards de control, asociados a cambios en el flujo de ejecución debidos a saltos y bifurcaciones; y hazards estructurales, causados por conflictos en el uso simultáneo de recursos hardware compartidos.(Patterson & Hennessy, 2017)

Análisis sistemático de técnicas de resolución de hazards Mnaaoui et al. (2018) estudian de forma sistemática distintas técnicas de resolución de hazards en un procesador RISC de 32 bits con pipeline de cinco etapas, descrito en VHDL y sintetizado en una FPGA Xilinx Virtex-5. El trabajo identifica y clasifica los hazards que aparecen en la arquitectura propuesta, diseña soluciones específicas para cada tipo e integra la lógica de gestión de hazards en la unidad de control del procesador.

Entre las técnicas implementadas se encuentran esquemas de forwarding mediante multiplexores adicionales en las entradas de la ALU para resolver dependencias de datos recientes, mecanismos de stalling que insertan burbujas cuando el reenvío no es suficiente, y una lógica de predicción y gestión de saltos que reduce la penalización asociada a los hazards de control. Estas técnicas se modelan y validan mediante simulaciones a nivel RTL, comparando el comportamiento del pipeline antes y después de activar las unidades de resolución de hazards.

Los autores muestran que la combinación de forwarding y stalling adecuadamente controlados permite reducir de forma significativa el número de ciclos de espera debidos a dependencias de datos, mientras que la mejora en el tratamiento de saltos disminuye el coste medio de las instrucciones de control sobre el CPI efectivo El análisis de síntesis en Virtex-5 indica que la lógica adicional de detección de hazards, multiplexores de reenvío y control de saltos introduce un incremento moderado en el uso de recursos lógicos, pero aporta una mejora clara en el rendimiento del procesador medida en términos de ciclos por instrucción,

lo que respalda la idea de que estas técnicas son costo-efectivas en arquitecturas pipeline.

7.2.11. Estudios comparativos multi-arquitectura

Los estudios que comparan procesadores RISC-V con arquitecturas alternativas como ARM o MIPS, así como los que evalúan múltiples implementaciones de núcleos RISC-V, proporcionan un contexto útil para situar el rendimiento esperado de diseños RV32IM dentro del espectro de procesadores RISC embebidos.(Heinz et al., 2019; Liu et al., 2021)

Comparación cuantitativa RISC-V vs. ARM vs. MIPS Liu et al. (2021) presentan un estudio de caso en el que comparan sistemas basados en RISC-V, ARM Cortex y MIPS bajo condiciones experimentales controladas, empleando un conjunto de benchmarks que incluye Dhrystone, Whetstone, CoreMark y MiBench. Un elemento clave de su metodología es la normalización de la frecuencia de operación para eliminar sesgos tecnológicos, de forma que las comparaciones se centran en métricas como CPI, número de instrucciones ejecutadas y rendimiento normalizado por MHz.

Los resultados muestran que RISC-V se sitúa generalmente en una posición intermedia: supera a ciertas configuraciones MIPS en algunas métricas de rendimiento normalizado, pero suele quedar por detrás de núcleos ARM de gama similar, que aprovechan técnicas más agresivas de resolución de hazards y optimizaciones microarquitecturales. Este tipo de estudios nos dice que, para ser competitivos, diseños RV32I/RV32IM de cinco etapas deben aspirar a CPI cercanos a 1 en cargas de trabajo típicas y a valores de CoreMark/MHz comparables a los de otros procesadores RISC embebidos de propósito general.

Impacto de la extensión M en el rendimiento Islam et al. (2020) analizan específicamente el impacto de la extensión M de RISC-V (instrucciones de multiplicación y división) sobre el rendimiento de un procesador de cinco etapas implementado como softcore en FPGA. Partiendo de una arquitectura base RV32I, proponen la variante RVCoreP-32IM con soporte hardware para la extensión M y evalúan distintas implementaciones de multiplicadores, como versiones iterativas basadas en radix-4 y variantes que explotan bloques DSP dedicados del FPGA.

Usando benchmarks como Dhrystone, CoreMark y Embench, muestran que la configuración RV32IM proporciona mejoras de rendimiento significativas respecto a RV32I para programas que utilizan instrucciones de multiplicación y división, y que el uso de multiplicadores basados en bloques DSP puede incrementar aún más la ganancia al combinar mayor frecuencia de operación con menor latencia para estas operaciones.(Islam et al., 2020) Al mismo tiempo, el estudio cuantifica el incremento de recursos lógicos y de bloques DSP asociado

a la extensión M, lo que evidencia que esta extensión introduce un overhead apreciable de área y complejidad que debe sopesarse frente a los *speedups* logrados en aplicaciones intensivas en operaciones aritméticas.

En el contexto de este proyecto, estos trabajos sugieren que un diseño RV32IM pipeline bien afinado debería buscar un CPI efectivo próximo a 1 en benchmarks mixtos y un rendimiento normalizado (por ejemplo, CoreMark/MHz) en el rango típico de los núcleos RISC-V embebidos analizados en la literatura, asumiendo un incremento razonable en LUTs y uso de DSPs frente a RV32I monociclo.(Islam et al., 2020; Liu et al., 2021)

7.2.12. Metodologías de verificación funcional

La verificación funcional rigurosa es crítica para garantizar que las implementaciones de procesadores ejecutan correctamente todas las instrucciones de la ISA según la especificación, especialmente al comparar arquitecturas como monociclo y pipeline, donde errores sutiles relacionados con hazards pueden pasar desapercibidos en pruebas superficiales.

Suite riscv-tests como referencia de conformidad La suite oficial riscv-tests, mantenida actualmente en el repositorio `riscv-software-src/riscv-tests`, se ha consolidado como referencia de facto para verificar conformidad ISA a nivel de instrucciones en procesadores RISC-V. Incluye grupos de pruebas dirigidos para conjuntos como RV32I y RV32IM, con decenas de tests unitarios que ejercitan instrucciones individuales y combinaciones básicas de operaciones, con informes de cobertura de más del 90 % para RV32I y cerca del 90 % para RV32IM según los análisis de Imperas.

Kumar y Singh (2023) aplican esta metodología a su procesador RV32I monociclo y reportan la ejecución correcta de todos los tests seleccionados de la suite, lo que establece un nivel de conformidad ISA exigente pero alcanzable en diseños académicos sobre FPGA. En el contexto de este proyecto, exigir que tanto la versión monociclo como la segmentada RV32IM superen el conjunto completo de pruebas de riscv-tests correspondiente se considera un requisito mínimo para que cualquier comparación de rendimiento entre ambas arquitecturas sea válida.

Cocotb como framework de verificación Cocotb es un framework de verificación basado en Python que permite escribir testbenches que interactúan con simuladores HDL como Icarus Verilog, Verilator o ModelSim mediante cosimulación, ofreciendo una alternativa flexible a entornos basados en SystemVerilog y UVM. (cocotb contributors, 2019) Un estudio comparativo reciente centrado en la verificación de un módulo AES-128 analiza Cocotb frente

a UVM en términos de productividad, cobertura y tiempo de simulación. (Susithra et al., 2024)

Los autores y el resumen técnico asociado destacan que, en el caso de estudio considerado, Cocotb logra una cobertura de código ligeramente superior y una cobertura funcional más completa que UVM, a costa de tiempos de simulación mayores debido al overhead de la cosimulación Python-HDL. Al mismo tiempo, el entorno Python y las bibliotecas asociadas reducen el esfuerzo necesario para construir el testbench y gestionar la sincronización de eventos, lo que se traduce en una mayor productividad en la creación y extensión de casos de prueba.

7.2.13. CoreMark como benchmark estándar para procesadores embebidos

CoreMark, desarrollado por el consorcio EEMBC, se ha consolidado como un benchmark estándar para evaluar el rendimiento de procesadores embebidos, sustituyendo progresivamente a Dhrystone gracias a un diseño que reduce la dependencia de la librería estándar y dificulta las optimizaciones no representativas del compilador.(Arm Limited, 2022; EEMBC, n.d.) El núcleo del benchmark combina procesamiento de listas enlazadas, operaciones matriciales, una máquina de estados finitos y cálculo de CRC, con datos generados en tiempo de ejecución para evitar que el compilador pueda precomputar resultados estáticos.(EEMBC, n.d.)

Las reglas de ejecución de CoreMark exigen que el programa se ejecute durante al menos 10 segundos para que los resultados sean válidos, y se recomienda realizar múltiples ejecuciones de perfilado y reportar el promedio junto con los parámetros de compilación utilizados.(Arm Limited, 2022; EEMBC, n.d.) Además de la puntuación absoluta, es habitual informar de la métrica normalizada CoreMark/MHz, que se obtiene dividiendo el resultado por la frecuencia efectiva y permite comparar la eficiencia arquitectural independientemente de la frecuencia de reloj del sistema.(Arm Limited, 2022; EEMBC, n.d.)

7.2.14. Identificación de la brecha en la literatura

A pesar de la proliferación de implementaciones RISC-V sobre FPGA descritas en la literatura, el análisis crítico de los trabajos revisados permite identificar varias limitaciones que dificultan extraer conclusiones sólidas sobre los trade-offs reales entre arquitecturas monociclo y pipeline de procesadores RV32I/RV32IM en FPGAs de uso educativo.(Heinz et al., 2019; «Performance-Optimised Design of the RISC-V Five-Stage Pipeline Processor», 2024; Reddy et al., 2024)

Ausencia de comparaciones controladas directas En la mayoría de estudios se implementa una única microarquitectura por trabajo, sin disponer de versiones monociclo y segmentada del mismo diseño evaluadas bajo el mismo flujo de síntesis y en la misma FPGA.(«Performance-Optimised Design of the RISC-V Five-Stage Pipeline Processor», 2024; Reddy et al., 2024) Por ejemplo, Reddy et al. (2024) presentan un procesador RV32I de ciclo único sobre Cyclone V, mientras que NRP describen un procesador RV32I pipeline de cinco etapas, pero ninguno de los dos trabajos compara directamente ambas variantes del mismo núcleo bajo condiciones experimentales idénticas.(«Performance-Optimised Design of the RISC-V Five-Stage Pipeline Processor», 2024; Reddy et al., 2024)

Como consecuencia, los resultados de distintos artículos no son directamente comparables, suelen diferir en la ISA exacta, en las optimizaciones microarquitecturales adoptadas, en la experiencia del equipo de diseño y en las herramientas y versiones de síntesis utilizadas, entre otros factores.(Heinz et al., 2019)

Heterogeneidad de plataformas FPGA Otra limitación relevante es la diversidad de plataformas utilizadas: algunos trabajos emplean FPGAs Xilinx Spartan-7 o Artix-7, otros usan dispositivos Intel Cyclone V u otras familias, cada una con arquitecturas internas y capacidades distintas.(Heinz et al., 2019; Nguyen & Le, 2024) Esta heterogeneidad implica que frecuencias máximas como 80–100 MHz en Artix-7 y del orden de 30 MHz en Cyclone V no pueden interpretarse de forma directa como evidencia de una mejora de 2–3 veces debida exclusivamente al uso de pipeline, ya que parte de la diferencia puede atribuirse a la tecnología y a la arquitectura del FPGA subyacente.(Chu, 2011)

Protocolos de medición inconsistentes Los estudios revisados también muestran una notable variación en los protocolos de medición y en las métricas reportadas.(Heinz et al., 2019; «Performance-Optimised Design of the RISC-V Five-Stage Pipeline Processor», 2024) Algunos trabajos utilizan CoreMark, otros se basan en Dhrystone u otros benchmarks específicos, mientras que en ciertos casos solo se informa de la frecuencia máxima o de un CPI medio sin acompañarlo de una carga de trabajo claramente definida ni de métricas normalizadas como CoreMark/MHz.(Arm Limited, 2022; EEMBC, n.d.)

Asimismo, las opciones de compilación y el nivel de detalle en la documentación de las condiciones de prueba difieren entre artículos, lo que dificulta integrar cuantitativamente resultados de distintas fuentes para extraer conclusiones robustas sobre los trade-offs arquitecturales.(Heinz et al., 2019; «Performance-Optimised Design of the RISC-V Five-Stage Pipeline Processor», 2024)

Ausencia de evidencia específica para DE1-SoC en contexto educativo Aunque la placa DE1-SoC basada en Intel Cyclone V es muy utilizada en entornos docentes de arquitectura de computadores, los trabajos que reportan resultados específicos para esta plataforma son escasos.(Nguyen & Le, 2024) En particular, la implementación de Reddy et al. (2024) proporciona datos para un procesador RV32I monociclo en Cyclone V, pero no se dispone de un estudio equivalente que presente y compare de forma sistemática una microarquitectura pipeline del mismo núcleo en la misma placa.

Esto deja a docentes e investigadores que trabajan con DE1-SoC sin datos empíricos directamente comparables que les ayuden a valorar, en condiciones controladas, si la complejidad adicional de una implementación pipeline RV32IM se justifica frente a un diseño monociclo en términos de rendimiento y uso de recursos.(Heinz et al., 2019; Reddy et al., 2024)

7.2.15. Conclusiones del estado del arte

El análisis de la literatura muestra que existen múltiples implementaciones monociclo y pipeline de procesadores RISC-V sobre FPGA, con resultados de rendimiento cuantificados en términos de frecuencia máxima, CPI efectivo y, en algunos casos, CoreMark/MHz.(Heinz et al., 2019; Kumar & Singh, 2023; «Performance-Optimised Design of the RISC-V Five-Stage Pipeline Processor», 2024; Reddy et al., 2024) En conjunto, los trabajos revisados sitúan a los núcleos RV32I monociclo de referencia en rangos típicos de decenas de MHz para la frecuencia máxima y CPI igual a 1, mientras que las microarquitecturas segmentadas de cinco etapas alcanzan frecuencias sensiblemente superiores y CPI efectivos algo mayores que 1 debido a los hazards, a la vez que demuestran mejoras claras en métricas normalizadas como CoreMark/MHz cuando se aplican técnicas adecuadas de resolución de hazards.(Heinz et al., 2019; Patterson & Hennessy, 2017; «Performance-Optimised Design of the RISC-V Five-Stage Pipeline Processor», 2024)

Sin embargo, no se ha identificado en la literatura un estudio que compare de forma controlada una arquitectura monociclo y una arquitectura pipeline del mismo procesador RV32IM implementadas en la misma FPGA y evaluadas bajo un protocolo de medición homogéneo, lo que impide extraer conclusiones directas sobre los trade-offs reales entre ambas opciones en un entorno educativo específico como DE1-SoC.(Heinz et al., 2019; Reddy et al., 2024)

A partir de esta revisión se destacan cuatro brechas principales:

(I) **Brecha metodológica:** Al no implementarse y evaluarse de forma conjunta ambas microarquitecturas del mismo núcleo bajo condiciones experimentales idénticas.

(ii) **Brecha plataforma-específica:** Debida a la escasez de datos detallados para la FPGA Intel Cyclone V utilizada en la tarjeta DE1-SoC.

(iii) **Brecha en los protocolos de medición:** Reflejada en la heterogeneidad de *benchmarks*, métricas y metodologías de verificación empleadas.

(iv) **Brecha a nivel de ISA:** Muchos trabajos se limitan al conjunto base RV32I, sin incorporar la extensión M pese a su relevancia en aplicaciones embebidas (Heinz et al., 2019; NRP, 2024; Reddy et al., 2024; RISC-V Foundation, 2019).

El presente proyecto se propone abordar estas brechas mediante una comparación experimental controlada entre versiones monociclo y pipeline de un mismo procesador RV32IM, implementadas sobre la FPGA de la DE1-SoC y evaluadas con protocolos de verificación y benchmarking claramente definidos, con el objetivo de aportar evidencia cuantitativa y reproducible sobre los trade-offs entre ambas arquitecturas.

8. Marco conceptual

8.1. Arquitectura de computadores

La arquitectura de computadores es la rama de la ingeniería informática que estudia la estructura y el comportamiento interno de los sistemas de procesamiento digital, incluyendo la organización del procesador, la memoria y los dispositivos de entrada/salida, así como la interfaz hardware-software que proporciona el conjunto de instrucciones.(Patterson & Hennessy, 2017) En particular, describe cómo se organizan el camino de datos, la unidad de control, la jerarquía de memoria y los mecanismos de comunicación con el exterior para ejecutar programas de manera eficiente.(Patterson & Hennessy, 2017)

En el contexto de este proyecto, la arquitectura determina cómo se estructuran e interconectan los componentes del procesador RISC-V RV32IM implementado en FPGA, tanto en su versión de ciclo único como en la versión segmentada de cinco etapas, condicionando el rendimiento, el uso de recursos y la complejidad de verificación.

8.2. Instruction Set Architecture (ISA)

La Instruction Set Architecture (ISA) es el conjunto formal de instrucciones, formatos de codificación binaria, comportamiento semántico y modelo de programación que un procesador expone al software.(Patterson & Hennessy, 2017; Waterman & Asanović, 2019) Actúa como interfaz entre hardware y software para compiladores, sistemas operativos y programadores, definiendo qué operaciones puede ejecutar el procesador, qué registros y modos de direccionamiento están disponibles y cómo se representan los datos, pero no cómo se implementan internamente estas operaciones.(Waterman & Asanović, 2019)

Es fundamental distinguir entre ISA y microarquitectura:

- **ISA (qué):** Especifica el conjunto de instrucciones, los registros visibles al programador, el formato de codificación y la semántica de cada operación. Por ejemplo, la ISA RV32IM define una instrucción `MUL rd, rs1, rs2` que multiplica los contenidos de `rs1` y `rs2` y almacena el resultado en `rd`.(Waterman & Asanović, 2019)

- **Microarquitectura (cómo):** Describe la organización interna del procesador que implementa esa ISA: estructura del datapath, presencia o no de pipeline, número y tipo de unidades funcionales, técnicas de forwarding, manejo de hazards, etc. Diferentes microarquitecturas pueden ejecutar la misma ISA con trade-offs distintos de rendimiento, área y consumo.(Patterson & Hennessy, 2017)

8.3. RISC-V y extensión M (RV32IM)

8.3.1. RISC-V como arquitectura abierta

RISC-V es una arquitectura de conjunto de instrucciones abierta, libre de regalías y modular, basada en principios RISC, que define un conjunto base mínimo y un conjunto de extensiones estándar que se pueden combinar según las necesidades de cada implementación.(Waterman & Asanović, 2019) Para enteros de 32 bits, el conjunto base RV32I ofrece 47 instrucciones distintas que abarcan operaciones aritméticas y lógicas, cargas y almacenamientos, comparaciones y control de flujo, y está diseñado para ser suficiente como objetivo de compilador en sistemas operativos modernos.(Waterman & Asanović, 2019)

Sobre este núcleo se definen extensiones opcionales identificadas por letras, como la extensión M para multiplicación y división enteras, la extensión A para operaciones atómicas, las extensiones F y D para punto flotante en simple y doble precisión, y la extensión C para instrucciones comprimidas de 16 bits, entre otras.(Waterman & Asanović, 2019)

8.3.2. Configuración RV32IM empleada en el proyecto

La configuración RV32IM combina el conjunto base entero RV32I con la extensión estándar M de multiplicación y división.(Waterman & Asanović, 2019) En RV32I se incluyen, entre otras, instrucciones aritméticas y lógicas sobre registros (por ejemplo, `ADD`, `SUB`, `AND`, `OR`, `XOR`), desplazamientos (`SLL`, `SRL`, `SRA`), comparaciones (`SLT`, `SLTU`), cargas y almacenamientos de distintos tamaños (`LB`, `LH`, `LW`, `SB`, `SH`, `SW`), saltos condicionales (`BEQ`, `BNE`, `BLT`, `BGE`), saltos incondicionales (`JAL`, `JALR`) y operaciones inmediatas como `ADDI`, `ANDI` u `ORI`.(Waterman & Asanović, 2019)

La extensión M añade ocho instrucciones de multiplicación y división de enteros, definidas para operandos de ancho XLEN (32 bits en RV32): `MUL`, `MULH`, `MULHU`, `MULHSU`, `DIV`, `DIVU`, `REM` y `REMU`.(Waterman & Asanović, 2019)

- `MUL rd, rs1, rs2`: realiza una multiplicación entero por entero y escribe en `rd` los XLEN bits menos significativos del producto.
- `MULH`, `MULHU` y `MULHSU`: devuelven los XLEN bits más significativos del producto para combinaciones signed×signed, unsigned×unsigned y signed×unsigned, respectivamente.
- `DIV`, `DIVU`: realizan división entera signed y unsigned, redondeando hacia cero.
- `REM`, `REMU`: proporcionan el resto de la división correspondiente, con reglas específicas para división por cero y casos de overflow.

8.4. Pipeline de procesadores

8.4.1. Definición y etapas

El pipeline es una técnica de organización microarquitectural que divide la ejecución de instrucciones en etapas secuenciales, de modo que varias instrucciones puedan encontrarse simultáneamente en distintas fases de ejecución, explotando paralelismo temporal para aumentar el throughput del procesador.(Harris & Harris, 2021; Patterson & Hennessy, 2017) En una arquitectura RISC clásica de cinco etapas, habitual en diseños MIPS y RISC-V, las fases son: búsqueda de instrucción, decodificación, ejecución, acceso a memoria y escritura de resultados.(**Ch6Pipeline2003**; Patterson & Hennessy, 2017)

Las etapas típicas de un pipeline de cinco niveles son:

- **IF (Instruction Fetch):** Obtención de la instrucción desde memoria de instrucciones usando el contador de programa (PC) y cálculo del siguiente PC.
- **ID (Instruction Decode / Register Fetch):** Decodificación de la instrucción, extracción de campos (opcode, registros fuente y destino, inmediatos) y lectura de operandos desde el banco de registros.
- **EX (Execute):** Ejecución de operaciones aritméticas o lógicas en la ALU, cálculo de direcciones efectivas para accesos a memoria y evaluación de condiciones de salto.
- **MEM (Memory Access):** Acceso a la memoria de datos para instrucciones de carga y almacenamiento; el resto de instrucciones atraviesan esta etapa sin operación sobre memoria.
- **WB (Write Back):** Escritura del resultado en el banco de registros para las instrucciones que actualizan un registro destino.

Entre cada par de etapas se insertan registros interetapa que almacenan señales de datos y de control de una instrucción mientras avanza por el pipeline, aislando temporalmente las etapas y permitiendo que cada una opere sobre una instrucción diferente en cada ciclo.(Patterson & Hennessy, 2017)

8.4.2. Ecuación de rendimiento y comparación monociclo vs. pipeline

El tiempo de ejecución de un programa puede expresarse mediante la ecuación clásica de rendimiento:

$$\text{Tiempo}_{\text{CPU}} = N_{\text{instr}} \times \text{CPI} \times T_{\text{ciclo}},$$

donde N_{instr} es el número de instrucciones ejecutadas, CPI el número medio de ciclos por instrucción y T_{ciclo} el tiempo de ciclo de reloj.(Patterson & Hennessy, 2017) A partir de esta relación, el throughput puede escribirse como:

$$\text{Throughput} = \frac{N_{\text{instr}}}{\text{Tiempo}_{\text{CPU}}} = \frac{f_{\text{reloj}}}{\text{CPI}} = f_{\text{reloj}} \times \text{IPC},$$

donde $f_{\text{reloj}} = 1/T_{\text{ciclo}}$ es la frecuencia de reloj e IPC es el número medio de instrucciones completadas por ciclo.

En un procesador monociclo, el CPI ideal es igual a 1, ya que cada instrucción se completa en un solo ciclo, pero el tiempo de ciclo debe dimensionarse según el camino crítico más largo, que normalmente incluye todas las operaciones necesarias para la instrucción más lenta del conjunto. En un procesador con pipeline de cinco etapas, el objetivo es mantener un CPI cercano a 1 en ausencia de hazards, mientras que el tiempo de ciclo se reduce al estar limitado por la etapa individual más lenta en lugar de por toda la ruta de datos combinacional, lo que permite aumentar la frecuencia y, por tanto, el throughput efectivo.(Patterson & Hennessy, 2017)

En la práctica, los hazards de datos, de control y estructurales incrementan el CPI por encima del ideal y reducen la ganancia teórica frente al monociclo, de modo que los incrementos observados de throughput suelen situarse entre un factor aproximado de 2 y 4 para pipelines de cinco etapas bien diseñados, según la carga de trabajo y las técnicas de resolución de hazards aplicadas. En este proyecto, la hipótesis de trabajo es que la microarquitectura pipeline RV32IM implementada en FPGA alcanzará un throughput superior al del diseño monociclo correspondiente, gracias a una frecuencia de operación más alta que compense el posible aumento de CPI debido a los hazards.

8.5. Hazards en pipelines de procesadores

Los hazards son situaciones que impiden la progresión ideal del pipeline y obligan a introducir mecanismos de detección y resolución para mantener la corrección funcional; su efecto es aumentar el CPI efectivo al provocar ciclos de espera o vaciados parciales del pipeline.(Harris & Harris, 2021; Patterson & Hennessy, 2017)

8.5.1. Tipos de hazards

Hazards de datos Un hazard de datos aparece cuando una instrucción necesita un operando que todavía no ha sido escrito en el banco de registros por una instrucción previa, lo que puede ocurrir con dependencias verdaderas de tipo read-after-write (RAW). En pipelines

sencillos in-order, las dependencias write-after-read (WAR) y write-after-write (WAW) no se manifiestan, ya que las escrituras y lecturas se ordenan de manera determinista.(Patterson & Hennessy, 2017)

Entre las técnicas de resolución, el forwarding, o bypassing, permite reenviar resultados desde etapas posteriores del pipeline, como EX/MEM o MEM/WB, directamente a las entradas de la ALU, evitando esperar a que el valor sea escrito en el banco de registros. Cuando el reenvío no es suficiente, por ejemplo en ciertos patrones de load-use, la unidad de detección de hazards inserta burbujas mediante stalling hasta que el dato está disponible, manteniendo la corrección funcional a costa de un aumento del CPI.(Mnaaoui et al., 2018; Patterson & Hennessy, 2017)

Hazards estructurales Los hazards estructurales se producen cuando dos o más instrucciones intentan usar simultáneamente un recurso hardware que no puede atenderlas a la vez, como una memoria o una unidad funcional compartida. Una forma habitual de evitarlos en procesadores RISC es emplear arquitecturas de tipo Harvard con memorias separadas de instrucciones y datos, reduciendo la probabilidad de conflicto entre la etapa IF y la etapa MEM; alternativamente, se pueden introducir mecanismos de arbitraje o stalling cuando el recurso está ocupado.(Chu, 2011; Patterson & Hennessy, 2017)

Hazards de control Los hazards de control aparecen cuando el flujo de instrucciones depende del resultado de un salto o branch que aún no se ha resuelto, de modo que las instrucciones ya presentes en el pipeline pueden no pertenecer al camino correcto de ejecución. Si no se aplica ninguna técnica de mitigación, la resolución de un salto puede requerir descartar varias instrucciones ya en el pipeline, introduciendo una penalización en ciclos cada vez que se toma un branch.(Harris & Harris, 2021; Patterson & Hennessy, 2017)

Para reducir este efecto, se emplean técnicas como la predicción de saltos, como por ejemplo, esquemas sencillos de *predict-not-taken* o predictores basados en contadores de saturación, así como la resolución temprana de branches en etapas anteriores del pipeline, con el objetivo de minimizar el número de ciclos perdidos por saltos mal predichos.(Mnaaoui et al., 2018; Patterson & Hennessy, 2017)

En ISAs modernas como RISC-V no se utilizan *branch delay slots*, a diferencia de lo que ocurría en algunos diseños MIPS clásicos, de modo que la lógica de control del pipeline debe encargarse explícitamente de anular las instrucciones especulativas cuando la predicción es incorrecta.(Waterman & Asanović, 2019)

8.5.2. Impacto de los hazards en el CPI

Desde el punto de vista del rendimiento, los hazards elevan el CPI efectivo por encima del valor ideal de 1, al introducir ciclos adicionales en los que el pipeline no completa nuevas instrucciones. Sin técnicas de resolución, la frecuencia de stalls y vaciados puede hacer que la ganancia de throughput respecto a un diseño monociclo se reduzca de forma significativa o incluso se anule en ciertos patrones de ejecución.(Patterson & Hennessy, 2017)

La literatura muestra que la combinación de forwarding bien diseñado, stalling selectivo y un tratamiento cuidadoso de los saltos permite mantener el CPI relativamente próximo a 1 para muchas cargas de trabajo, preservando las ventajas del pipeline respecto al monociclo, especialmente cuando se acompaña de una frecuencia de reloj superior.(Mnaaoui et al., 2018; «Performance-Optimised Design of the RISC-V Five-Stage Pipeline Processor», 2024)

8.5.3. Field-Programmable Gate Array (FPGA)

Una FPGA es un circuito integrado reprogramable por el usuario tras la fabricación, formado por bloques lógicos configurables, recursos de memoria embebida y una red de interconexión programable. Estos dispositivos permiten implementar arquitecturas digitales complejas, como procesadores, a partir de descripciones en lenguajes de hardware y herramientas de síntesis y place&route.(Author, 2010; Chu, 2011)

8.5.4. Componentes FPGA relevantes

Desde el punto de vista de este proyecto, los recursos más relevantes de una FPGA son:

- **LUTs (Look-Up Tables):** Bloques lógicos configurables que implementan funciones booleanas de varias entradas; el número disponible condiciona la capacidad lógica total del dispositivo.(Chu, 2011)

- **Flip-flops (FFs):** Elementos de almacenamiento secuencial utilizados para registrar señales y construir registros y máquinas de estados; los pipelines tienden a requerir más flip-flops que los diseños monociclo debido a los registros interetapa.(Chu, 2011)

- **Memoria embebida (Block RAM):** Bloques de memoria dedicados utilizados para implementar memorias de instrucciones, de datos y estructuras como bancos de registros de forma más eficiente que con LUTs.(Author, 2010)

- **Bloques DSP:** Unidades aritméticas dedicadas, optimizadas para operaciones de multiplicación y acumulación, que resultan especialmente útiles para implementar de forma eficiente instrucciones de la extensión M de RISC-V.(Intel Corporation, 2018)

8.5.5. FPGA DE1-SoC (Cyclone V 5CSEMA5F31C6)

La placa DE1-SoC emplea un dispositivo Intel Cyclone V SoC modelo 5CSEMA5F31C6, que integra lógica programable y un subsistema de procesador ARM Cortex-A9.(Intel Corporation, 2016; Terasic Technologies, 2013) Según las especificaciones del fabricante, este dispositivo ofrece aproximadamente 85 000 elementos lógicos programables, 4 450 Kbits de memoria embebida y bloques especializados para operaciones DSP.(Intel Corporation, 2016; Terasic Technologies, 2013) Estos recursos permiten implementar procesadores suaves (soft-cores) como el RISC-V RV32IM junto con las memorias de instrucciones y datos necesarias en el propio tejido de la FPGA.

En este proyecto se utiliza el flujo de diseño proporcionado por Intel Quartus Prime para sintetizar, implementar y analizar las versiones monociclo y pipeline del procesador sobre la FPGA de la DE1-SoC, aprovechando los informes de utilización de recursos y de frecuencia máxima para cuantificar el uso de LUTs, registros, memoria y bloques DSP.(Chu, 2011; Intel Corporation, 2016)

8.6. SystemVerilog y diseño RTL

SystemVerilog es un lenguaje de descripción de hardware que unifica capacidades de diseño, especificación y verificación, extendiendo Verilog con tipos de datos más ricos, mecanismos orientados a objetos y construcciones específicas para verificación como assertions y cobertura. El estándar IEEE 1800-2017 define la sintaxis y semántica de SystemVerilog, e indica que el lenguaje soporta modelado de hardware a nivel comportamental, de transferencia de registros (RTL) y de puertas, así como la escritura de testbenches con técnicas de verificación avanzada.(«IEEE Standard for SystemVerilog (IEEE 1800-2017)», 2017)

8.6.1. Nivel RTL

En el nivel de transferencia de registros (RTL), el diseño se describe en términos de registros y de las transferencias de datos entre ellos sincronizadas por señales de reloj, especificando el comportamiento ciclo a ciclo sin detallar la implementación física a nivel de puertas individuales. Este nivel de abstracción es el habitual para describir procesadores y otros bloques complejos de hardware digital, ya que permite capturar claramente el datapath, la unidad de control y las interfaces, manteniendo la compatibilidad con las herramientas de síntesis para FPGA y ASIC.(Chu, 2011)

8.6.2. Proceso de síntesis sobre FPGA

El flujo típico de implementación de un diseño RTL en FPGA consta de varias etapas. En primer lugar, el comportamiento del procesador se describe mediante código SystemVerilog a nivel RTL, incluyendo el camino de datos y la lógica de control para las versiones monociclo y pipeline del núcleo RV32IM.(Chu, 2011)
A continuación, la herramienta de síntesis traduce la descripción RTL a una netlist de recursos específicos de la FPGA, como LUTs, flip-flops, bloques de memoria y bloques DSP, aplicando optimizaciones según las restricciones de tiempo y área.

Posteriormente, las fases de place & route colocan físicamente los recursos lógicos en el dispositivo y determinan las rutas de interconexión entre ellos, respetando las restricciones de temporización establecidas por el diseñador.(Intel Corporation, 2016)

Finalmente, el análisis estático de temporización (Static Timing Analysis, STA) evalúa todos los caminos de datos relevantes y calcula la frecuencia máxima de operación $f_{\max}$ que el diseño puede alcanzar sin violar las restricciones de tiempo de establecimiento y hold, proporcionando una medida objetiva del límite de frecuencia para cada implementación.(«IEEE Standard for SystemVerilog (IEEE 1800-2017)», 2017; Intel Corporation, 2016)

8.7. Verificación funcional y suites de prueba

8.7.1. riscv-tests como suite de conformidad

La suite riscv-tests, mantenida por la comunidad RISC-V, proporciona conjuntos de pruebas dirigidos para verificar la conformidad de implementaciones de la ISA a nivel de instrucciones. Incluye, entre otros, grupos de tests para el conjunto base entero en modo usuario y para la extensión M de multiplicación y división, implementados como programas en ensamblador que ejercitan sistemáticamente las instrucciones correspondientes.(**riscvTestsRepo**)

Cada test ejecuta una secuencia de instrucciones y almacena resultados en ubicaciones de memoria o registros previamente acordadas; el test se considera superado si, al finalizar la ejecución, estos valores coinciden con los resultados esperados definidos en el propio código de prueba.(**riscvTestsRepo**; **AboutRiscvTesting**)

9. Presupuesto

Este proyecto de pregrado no cuenta con financiación externa ni interna específica asociada, por lo que el presupuesto asignado es igual a $0.

Referencias

Analog Devices. (2023). *CoreMark® user's guide for ADuCM3027/ADuCM3029* (Application note N.º EE-389). Analog Devices. https://www.analog.com/media/en/technical-documentation/application-notes/EE-389.pdf

Anjana, K. M., Anusha, S. B., Dikshitha, U., & Shilpa, V. (2025). Implementation of RISC-V Single Cycle Core. *International Journal of Advanced Research in Computer and Communication Engineering*, *14*(1).

Arm Limited. (2022). *CoreMark benchmarking for ARM Cortex processors* (Application note). Arm Limited. https://documentation-service.arm.com/static/6331cea74c59b30b51770bca?token=

Author, N. (2010). *FPGAs & Synthesizable Verilog* (inf. téc.). MIT CSAIL. https://projects.csail.mit.edu/beehive/FPGA_Verilog.pdf

Chu, P. P. (2011). *FPGA Prototyping by SystemVerilog Examples*. Wiley.

cocotb contributors. (2019). cocotb – Coroutine based cosimulation library for writing VHDL and Verilog testbenches in Python [Accessed 2026-03-01].

EEMBC. (2009). CoreMark - CPU Benchmark [Embedded Microprocessor Benchmark Consortium].

EEMBC. (n.d.). *CoreMark® – Embedded microprocessor benchmark*. https://www.eembc.org/coremark/

Harris, S. L., & Harris, D. M. (2021). *Digital Design and Computer Architecture: RISC-V Edition*. Morgan Kaufmann.

Heinz, C., Lavan, Y., Hofmann, J., & Koch, A. (2019). A Catalog and In-Hardware Evaluation of Open-Source Drop-In Compatible RISC-V Softcore Processors. *Proceedings of the 2019 29th International Conference on Field-Programmable Logic and Applications (FPL)*, 153-159. https://doi.org/10.1109/FPL.2019.00032

IEEE Standard for SystemVerilog (IEEE 1800-2017). (2017). IEEE Standards Association.

Intel Corporation. (2016). *Cyclone V Device Handbook: Volume 1: Device Interfaces and Integration*.

Intel Corporation. (2018). *Variable-precision DSP block in Cyclone® V devices*. Application note / Device handbook section. https://www.intel.com/content/www/us/en/docs/programmable/683694/current/variable-precision-dsp-block.html

Islam, M., et al. (2020). RVCoreP-32IM: An Effective Architecture to Implement mul/div Instructions for Five Stage RISC-V Soft Processors [Preprint arXiv:2010.16171]. *Journal of Supercomputing or similar*. https://arxiv.org/abs/2010.16171

Kumar, R., & Singh, R. (2023). Implementation of FPGA based 32-bit RISC-V Processor. *Microprocessors and Microsystems*, *98*, 104789. https://doi.org/10.1016/j.micpro.2023.104789

Liu, Y., Shao, Z., & Zhang, F. (2021). Performance Evaluation of Various RISC Processor Systems. *ACM International Conference Proceeding Series*, 24-29. https://doi.org/10.1145/3475992.3476003

Mnaaoui, S., Karray, F., & Masmoudi, N. (2018). Pipeline Hazards Resolution for a New Programmable Processor. *International Journal of Advanced Computer Science and Applications*, *9*(10), 380-387. https://doi.org/10.14569/IJACSA.2018.091051

Murray, K. E., Petelin, O., Zhong, S., Wang, J. M., Eldafrawy, M., Legault, J.-P., & Betz, V. (2020). VTR 8: High-performance CAD and customizable FPGA architecture modelling. *ACM Transactions on Reconfigurable Technology and Systems*, *13*(2), 1-55. https://doi.org/10.1145/3388617

Nguyen, H. T., & Le, T. H. (2024). Evaluation of Softprocessor RISC-V for Edge Computing Applications. *IEEE Region 10 Symposium (TENSYMP)*, 1-6. https://doi.org/10.1109/TENSYMP58410.2024

NRP. (2024). Performance-Optimised Design of the RISC-V Five-Stage Pipeline Processor. *International Journal of Advanced Computer Science and Applications (IJACSA)*, *15*(2). https://doi.org/10.14569/IJACSA.2024.01502106

Patterson, D. A., & Hennessy, J. L. (2017). *Computer Architecture: A Quantitative Approach* (6th). Morgan Kaufmann.

Percey, A. (2007, diciembre). *Advantages of the Virtex-5 FPGA 6-Input LUT Architecture* (White paper N.º WP284 (v1.0)). Xilinx. https://www.bdtic.com/download/XILINX/wp284.pdf

Performance-Optimised Design of the RISC-V Five-Stage Pipeline Processor. (2024). *International Journal of Advanced Computer Science and Applications*, *15*(2), 278-285. https://doi.org/10.14569/IJACSA.2024.0150230

Ramos-Hernandez, D. N., & Tokhi, M. O. (2003). Comparison of DSP, RISC and transputer based systems. *Systems Analysis Modelling Simulation*, *43*(6), 805-822.

Reddy, A., Kumar, S., Reddy, M. V., & Rao, K. S. (2024). Implementation of RISC-V Single Cycle Core. *International Journal of Advanced Research in Computer and Communication Engineering*, *13*(4), 127-133.

RISC-V Foundation. (2019). *riscv-tests: RISC-V ISA tests*. GitHub. https://github.com/riscv/riscv-tests

ScienceDirect Topics. (2024). Pipeline Processor - An Overview. En *Computer Architecture Reference*. Elsevier.

Smith, J. E., & Pleszkun, A. R. (1988). Implementing Precise Interrupts in Pipelined Processors. *IEEE Transactions on Computers*, *37*(5), 562-573. https://doi.org/10.1109/12.2207

Susithra, N., Kanna S, S., Karthik K, S., Raja T, N., Yaswant, V., & Rajalakshmi, K. (2024). Analyzing AES Verification: A Comparative Study of UVM and Cocotb Approaches. *2024 International Conference on Smart Systems for Electrical, Electronics, Communication and Computer Engineering (ICSSEECC)*, 478-482. https://doi.org/10.1109/ICSSEECC61126.2024.10649439

Terasic Technologies. (2013). *DE1-SoC Development and Education Kit*. https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&CategoryNo=167&No=836

Wall, D. W. (1991). *Limits of Instruction-Level Parallelism* (inf. téc. N.º WRL Research Report 91/15). Digital Equipment Corporation, Western Research Laboratory. https://www.hpl.hp.com/techreports/Compaq-DEC/WRL-91-15.pdf

Waterman, A., & Asanović, K. (2019). *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA, Version 2.2*. RISC-V Foundation.

Xilinx. (2014). *7 series DSP48E1 slice user guide*. User guide. https://docs.amd.com/v/u/en-US/ug479_7Series_DSP48E1

Xilinx. (2020). *7 Series FPGAs Memory Resources: User Guide*. User guide.

Yi, L., Li, C., & Guo, J. (2020). CPI for Runtime Performance Measurement: The Good, the Bad, and the Ugly. *IEEE International Symposium on Workload Characterization (IISWC 2020)*, 15-25. https://doi.org/10.1109/IISWC50251.2020.00011